

TIA Portal Openness: API for automation of engineering workflows

Manual

20.00.00.00

01/2025

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

 DANGER
indicates that death or severe personal injury will result if proper precautions are not taken.
 WARNING
indicates that death or severe personal injury may result if proper precautions are not taken.
 CAUTION
indicates that minor personal injury can result if proper precautions are not taken.
NOTICE
indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

 WARNING
Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient

conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

01/2025 Subject to change

Copyright © Siemens 2024.
All rights reserved

Table of content

1	Importing a screen with a faceplate instance	7
2	Importing/exporting data of a PLC device	10
2.1	Blocks	10
2.1.1	XML structure of the block interface section	10
2.1.2	Changes of the object model and XML file format	19
2.1.3	Exporting DBs with snapshots	20
2.1.4	Exporting blocks with know-how protection	23
2.1.5	Export/Import of SCL blocks	23
2.1.6	Export/Import of structured types of SCL blocks	40
2.1.7	Export/Import of SCL call blocks	47
2.1.8	Export/Import multilingual comments in SCL	61
2.1.9	Exporting failsafe blocks	62
2.1.10	Exporting system blocks	62
2.1.11	Exporting GRAPH blocks with multi-language text	63
2.1.12	Importing block	65
2.1.13	Exporting blocks	66
2.1.14	Importing blocks/UDT with open reference	76
2.1.15	Importing blocks/UDT for structural change object	78
2.1.16	Export/Import of unit specific publishing attribute of blocks and types	79
2.1.17	Creating Instance DB	84
2.1.18	Accessing DB value parameter without export	85
2.1.19	Export/Import of Plc Alarm TextLists	89
2.1.20	Export Document information to SimaticML file	96
2.1.21	Export/Import of Alarm Instance Text	97
2.1.22	Export/Import of Alarm classes	100
2.1.23	Export/Import of global supervision for ProDiag-FB	104
2.1.24	Export/import of ProDiag supervisions from Global overview editor	106
2.1.25	Export/Import of settings for ProDiag supervisions	108
2.1.26	Export/Import Watch & Force Table	111
2.1.27	Exporting user data type	113
2.1.28	Importing user data type	113
2.1.29	Exporting program block as document	115
2.1.30	Importing program block from document	117
2.1.31	Exporting UDT as document	119
2.1.32	Export of data in OPC UA XML format	121
2.1.33	Importing UDT from document	122
2.1.34	Export/Import of UDT & DB	0
2.1.35	Export/Import of Array & Instance DB	0
2.2	Tag tables	0

2.2.1	Exporting PLC tag tables	0
2.2.2	Importing PLC tag table	0
2.2.3	Exporting an individual tag or constant from a PLC tag table	0
2.2.4	Importing an individual tag or constant into a PLC tag table	0

Importing a screen with a faceplate instance

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project

Application

The following data of a faceplate instance in a screen is imported:

Screen	Data
Attributes	Left, Top, Width, Height, ObjectName, Resizing, TabIndex, FaceplateTypeName
Interface Attributes	All configured interface attributes of a faceplate instance are imported for importable screen items.
Compositions	• Animations All movement animations are imported. Tag animations rely on interface attributes. • Events All configured events are imported.

The existence of the following attributes is mandatory for the import:

- ObjectName
- FaceplateTypeName

The following data of inner screen items of a faceplate instance is excluded from the export and import:

Screen item	Attribute
IO-Field	Flashing on limit violation

Graphic IO-Field	Fit embedded graphic object to screen size
------------------	--

Restrictions

- Unknown Faceplate, event or interface attribute

If a faceplate type name, an event name or an interface attribute name is specified in the import file which does not exist in the project, the import is aborted with an Exception.

- Resizing behavior of a faceplate instance

The attribute "Resizing" is imported in any case, independent of the export options.

Examples:

If "Resizing" is set to "KeepRatio", the "Height" attribute is used to calculate the "Width" attribute value.

- The size of a faceplate type is 100 x 100 pixel. If a faceplate instance is imported with size 300 x 100 pixel and value "FixedSize" is set for the "Resizing" attribute, the import succeeds and the faceplate size is set to 100 x 100 pixel.
- The size of a faceplate type is 100 x 50 pixel. A faceplate instance is imported with size 100 x 100 pixel and value "KeepRatio" is set for the "Resizing" attribute. The import succeeds and the faceplate size is set to 200 x 100 pixel.

Note

Sizing behavior of imported faceplate instances

The values of "Resizing" and values of interface attributes can affect the size of the imported faceplate instance and even the size of the inclosed screen items.

To avoid unrequested changes of the appearance of a faceplate instance, import a faceplate with the initial size or even without "Width" and "Height" attribute values.

- Deviant interface attribute values

- If you modify attributes for the import, the last applied interface attribute value is imported.
- If attributes depend on each other, other attribute values can be changed during the import.

Example: A faceplate includes an I/O field. The attribute "Mode" is connected to an interface attribute. If you first set the mode to "Output" and then set the attribute "Hidden input" to true, the value of "Hidden input" is not applied after import. The first modification set the attribute "Hidden input" to read-only and therefore the value cannot be applied.

- If a attribute value does not fulfill the restrictions of WinCC, the faceplates type value is displayed.

Example: The display range of a gauge is set from 10 - 80. The attributes "MaximumValue" and "MinimumValue" are configured at interface attributes. If you set a minimum value that exceeds the maximum value, e. g. 100, the faceplate type's value for "MinimumValue" is displayed after import.

- If an interface attribute is connected with several screen item attributes within the faceplate type, the interface attribute value at the faceplate instance will display the applied attribute value of the first connected screen item.

Example: A faceplate includes two gauge objects with deviant maximum values. The minimum values of both gauges are connected to one single interface attribute.

If you first set a minimum value that is applicable for both gauges, both values are set.

If you set then a value that is only applicable for the second gauge, the value is only set for the second gauge, but the value of the first gauge is displayed as interface attribute.

Program code: Importing screens including a faceplate instance

Modify the following program code to import a screen including a faceplate instance:

 Copies the following program code to clipboard.

```
//Imports single screen including a faceplate instance
private static void
ImportSingleScreenWithFaceplateInstance(HmiTarget hmitarget)
{
    FileInfo info = new
    FileInfo(@"D:\Samples\Screens\ScreenFaceplate.xml");
    hmitarget.ScreenFolder.Screens.Import(info,
    ImportOptions.None);
}
```

Importing/exporting data of a PLC device

2.1 Blocks

2.1.1 XML structure of the block interface section

Basic principle

The data in the export file from the import/export is structured with reference to a basic structure. Every import file has to fulfill the basic structural conditions.

The export file includes all edited tags and constants of the interface section of an exported block. All attributes with "ReadOnly="TRUE"" and "Informative="TRUE"" are excluded.

If the information is redundant, it must exactly be identical in the import XML file and the project data. Otherwise the import will throw a recoverable exception.

The project data can contain more data than the import XML file, e. g. an external type may have additional members

Only writeable values can be imported via TIA Portal Openness XML.

Depending on the TIA Portal Openness export settings, the export file includes a defined set of attributes and elements. The XML exported from higher versions of the product is not compatible during the import operation in lower version of the TIA Portal.

Basic structure

The interface section of an exported block is covered in the `<Interface>` element in the SimaticML of a block. The root object is the `<Sections>` element, which represents the interface section of an exported block. The sequence of the following description of elements represents the required sequence in the input file.

```

<Interface>
  <Sections xmlns="http://www.siemens.com/automation/Openness/SW/Interface/v1">
    <Section Name="Input">
      <Member Name="input1" Datatype="Bool" Remanence="Volatile" Accessibility="Public">
        <AttributeList>
          ...
        </AttributeList>
      </Member>
      <Member Name="input2" Datatype="Bool" Remanence="Volatile" Accessibility="Public">
        <AttributeList>
          ...
        </AttributeList>
      </Member>
    </Section>
    <Section Name="Output">
      <Member Name="output1" Datatype="Bool" Remanence="Volatile" Accessibility="Public">
        <AttributeList>
          ...
        </AttributeList>
      </Member>
    </Section>
    <Section Name="InOut" />
    <Section Name="Static" />
    <Section Name="Temp" />
    <Section Name="Constant" />
  </Sections>
</Interface>

```

- Section

Section represents a single parameter or local data of a program block

- Member

Member represents the tags or constants used in the program block. Depending of the datatype of a tag, members can be nested or have further structural sub elements.

In case of the data type "ARRAY" the structural element "Subelement Path" represents e. g. the index of the components of an array element.

Only those members are exported, which were edited by the user.

- AttributeList

The <AttributeList> includes all defined attributes of a member. Attributes, that are system defined or assigned by a standard value are not listed in the XML structure.

The member attributes <ReadOnly> and <Informative> are only written to the XML export file if their value is TRUE.

- StartValue

The element <StartValue> is only written, if the default value of the tag or constant is set by the user.

```

...
<Member Name="Static_1" Datatype="Int">
  ...
  <StartValue>1</StartValue>
</Member>
...

```

- Comment

The element `<Comment>` is written, if it is set by the user. Comments of a tag or constant are exported as multilingual text:

```
...
<Member Name="Static_3" Datatype="Struct">
  <AttributeList>
    <BooleanAttribute Name="ExternalAccessible" SystemDefined="true">false</BooleanAttribute>
    <BooleanAttribute Name="ExternalVisible" SystemDefined="true">false</BooleanAttribute>
    <BooleanAttribute Name="ExternalWritable" SystemDefined="true">false</BooleanAttribute>
  </AttributeList>
  <Comment>
    <MultiLanguageText Lang="de-DE">An individual comment</MultiLanguageText>
  </Comment>
</Member>
...
```

Attributes

- Main attributes

The main attributes are written in the `<Member>` element in the XML structure.

```
...
<Member Name="Static_1" Datatype="User_data_type_1" Remanence="Retain">
  ...
</Member>
...
```

The following table shows the main attributes of a tag or constant at the block interface section.

Name	Datatype	Default	Import condition	Comment
Name	STRING	-	Required	
Datatype	ENUM	-	Required	
Version	STRING	-	Optional	
Remanence	ENUM	NonRetain	-	only written if not default
Accessibility	ENUM	Public	-	pre-defined by the system cannot be changed by the user
Informative	BOOL	FALSE	-	

Members with the flag "Informative" are ignored during import. If the attribute is deleted or set to FALSE, an exception is thrown.

Note

Remanence settings "Set in IDB"

If the remanence value of a tag or constant is "Set in IDB", the remanence set in the IDB has to be the same for all other tags and constants with the remanence value "SetInIDB".

The first imported member with "Set in IDB" attribute defines the expected remanence in the IDB for the following tags and constants with the remanence value "SetInIDB".

- System defined member attributes

Systemdefined member attributes are listed in the element <AttributeList>.

Systemdefined member attributes flagged with the <Informative> and are ignored during import.

```
...
<Member Name="Static_3" Datatype="Struct">
  <AttributeList>
    <BooleanAttribute Name="ExternalAccessible" SystemDefined="true">false</BooleanAttribute>
    <BooleanAttribute Name="ExternalVisible" SystemDefined="true">false</BooleanAttribute>
    <BooleanAttribute Name="ExternalWritable" SystemDefined="true">false</BooleanAttribute>
  </AttributeList>
  <Comment>
    <MultiLanguageText Lang="de-DE">An individual comment</MultiLanguageText>
  </Comment>
</Member>
...
```

UserDeletable	bool	TRUE	TRUE	Editor does not allow to delete the member
HmiAccessible	bool	TRUE	FALSE	No HMI access, no structure item
HmiVisible	bool	TRUE	FALSE	Filter to reduce the number of members shown in the first place
Offset	int	-	TRUE	DB, FB, FC (Temp). For classic PLCs and for Plus PLCs where the remanence is set to classic.
PaddedSize	int	-	TRUE	DB, FB, FC (Temp). For classic PLCs and for Plus PLCs where the remanence is set to classic. Only for arrays.
HiddenAssignment	bool	FALSE	FALSE	Hide assignment at call if matches with PredefinedAssignment
PredefinedAssingment	string	""	FALSE	Input for the paramter used when call is placed
ReadOnlyAssignment	bool	FALSE	FALSE	The user cannot change the predefined assignement at the call
UserVisible	bool	TRUE	TRUE	This member is not shown on the UI
HmiReadOnly	bool	TRUE	TRUE	This member is read only for HMI
CodeReadOnly	bool	FALSE	TRUE	-

- User defined attributes

User defined attributes are flagged with <ReadOnly>. Members with this flag are ignored during import. If the flag is deleted or set to FALSE, an exception is thrown.

Unedited user defined attributes are excluded from the export.

Name	Type	Default	SimaticML ReadOnly (informative)	Comment
CFC	IBlockAttribute	---	FALSE	this is a Payload

Datatype "STRUCT"

The components of the datatype "STRUCT" are represented in the XML structure of an import/export file as nested members:

```
<Sections xmlns="http://www.siemens.com/automation/Openness/SW/Interface/v2">
  <Section Name="Static">
    <Member Name="Static_1" Datatype="Struct">
      <!-- Basic struct -->
      <Member Name="Static_1" Datatype="Int">
        <!-- First Member of struct -->
        <StartValue>1</StartValue>
      </Member>
      <Member Name="Static_2" Datatype="Int">
        <!-- Second Member of struct -->
      </Member>
      <Member Name="Static_3" Datatype="Struct">
        <!-- A subsequent struct -->
        <Member Name="Static_1" Datatype="Int">
          <!-- First Member of the subsequent struct -->
          <StartValue>3</StartValue>
        </Member>
        <Member Name="Static_2" Datatype="Int">
          <!-- Second Member of the subsequent struct -->
        </Member>
      </Member>
    </Member>
  </Section>
</Sections>
```

Datatype "ARRAY" basic type

The components of the basic datatype "ARRAY" are represented in the XML structure of an import/export file as subelements with the attribute "Path" :

```

<Sections xmlns="http://www.siemens.com/automation/Openness/SW/Interface/v2">
  <Section Name="Static">
    <Member Name="Static_1" Datatype="Array[0..2] of Int" Remanence="Retain">
      <!-- Basic Array -->
      <Subelement Path="0">
        <!-- First Array Component-->
        <StartValue>1</StartValue>
      </Subelement>
      <Subelement Path="1">
        <!-- Second Array Component-->
        <StartValue>2</StartValue>
      </Subelement>
    </Member>
  </Section>
</Sections>

```

Datatype "ARRAY" of UDT

The components of the datatype "ARRAY" of an UDT are represented in the XML structure of an import/export file as new <sections> element in a <member> element. The members in the new section for UDT are within an ARRAY are assigned as subelements with "Path" attribute:

```

<Sections xmlns="http://www.siemens.com/automation/Openness/SW/Interface/v2">
  <Section Name="Static">
    <Member Name="Static_1" Datatype="&quot;User_data_type_1&quot;" Remanence="Retain">
      <Sections> <!-- Sections including the UDT "User_data_type_1" -->
        <Section Name="None">
          <Member Name="Element_2" Datatype="Int">
            <StartValue>47</StartValue>
          </Member>
        </Section>
      </Sections>
    </Member>
    <Member Name="Static_2" Datatype="Array[0..1] of &quot;User_data_type_1&quot;">
      <Sections> <!-- Sections including the UDT "User_data_type_1" -->
        <Section Name="None">
          <Member Name="Element_2" Datatype="Int">
            <Subelement Path="0"> <!-- Component of the array -->
              <StartValue>123</StartValue>
            </Subelement>
          </Member>
        </Section>
      </Sections>
    </Member>
  </Section>
</Sections>

```

Datatype "ARRAY" in "ARRAY"

The components of the datatype "ARRAY" in another ARRAY are represented in the XML structure of an import/export file as subelements with the attribute "Path".

The members within another ARRAY are assigned as subelements with "Path" attribute, if the component is edited by the user:

```

<Sections xmlns="http://www.siemens.com/automation/Openness/SW/Interface/v2">
  <Section Name="Static">
    <Member Name="Static_1" Datatype="Array[0..2] of Struct">
      <Member Name="Static_1" Datatype="Int" />
      <Member Name="Static_2" Datatype="Array[0..1, 0..3, -9..-2] of Struct">
        <Member Name="Static_1" Datatype="Int">
          <Subelement Path="0,0,3,-5">
            <StartValue>1</StartValue>
          </Subelement>
        </Member>
        <Subelement Path="0,0,2,-6">
          <Comment>
            <MultiLanguageText Lang="de-DE">A individual comment</MultiLanguageText>
          </Comment>
        </Subelement>
      </Member>
    </Member>
  </Section>
</Sections>

```

PLC data types (UDT)

The XML structure of a PLC data type depends on the TIA Portal Openness export settings.

- `ExportOptions.None`

Members of PLC data type are only written if the default value of at least one of the components is set by the user. For these members, only the two additional attributes "Name" and "Datatype" are written, to identify the member to which the `<StartValue>` belongs. Other members and attributes are not written.

- `ExportOptions.WithDefaults`

The following attributes are always written:

- Name
- Datatype
- ExternalAccessible
- ExternalVisible
- ExternalWritable
- SetPoint
- StartValue

Only written to the XML if the default value in this type is set by the user. If it only has been set in the PLC data type, it is not written.

- `ExportOptions.ReadOnly`

For PLC data types this setting will not lead to meaningful result. In combination with other settings it will have no influence on the result.

Overlaid tags

If a tag is overlaid with a new datatype, the members are represented in the XML structure of the new data type. The following XML structure shows a datatype WORD overlaid by an ARRAY of BYTE.

```
<Sections xmlns="http://www.siemens.com/automation/Openness/SW/Interface/v2">
  <Section Name="Input" />
  <Section Name="Output" />
  <Section Name="InOut" />
  <Section Name="Static">
    <Member Name="Static_1" Datatype="Word">
      <!-- Basic Member -->
      <StartValue>16#17</StartValue>
    </Member>
    <Member Name="Static_2" Datatype="Array[0..1] of Byte">
      <AttributeList>
        <StringAttribute Name="At" SystemDefined="true">Static_1</StringAttribute>
      </AttributeList>
      <!-- The AT member -->
      <Subelement Path="0">
        <!-- First overlay byte -->
      </Subelement>
      <Subelement Path="1">
        <!-- Second overlay byte -->
      </Subelement>
    </Member>
  </Section>
  <Section Name="Temp" />
  <Section Name="Constant" />
</Sections>
```

Block Interface

All attributes with `ReadOnly="TRUE"` and `Informative="FALSE"` are excluded. The XML structure of a block interface depends on the TIA Portal Openness export settings.

- `ExportOptions.None`

This setting exports only the modified data or the data that differs from the default.

In case their attribute definition does not specify a default value, the attribute is always written.

The export file also contains all values that are obligatory for the subsequent data import.

- `ExportOptions.WithDefaults`

The following attributes are always written

- Name
- Datatype
- `HmiAccessible` exported as `ExternalAccessible`
- `HmiVisible` exported as `ExternalVisible`
- `ExternalWritable`
- `SetPoint` (if applicable)

- Offset (if applicable)
- PaddedSize (if applicable)

All other attributes are only written if their values differ from the default.

The <StartValue> element is only written to the XML if it has been explicitly set.

- `ExportOptions.ReadOnly`

For block interfaces this setting will not lead to meaningful result. In combination with other settings it will have no influence on the result.

2.1.2 Changes of the object model and XML file format

Introduction

To import a custom created or an edited XML file successfully to the TIA Portal via TIA Portal Openness, the file must correspond to defined schemas.

The XML files always consist of two major parts:

- Interface
- Compile unit

The schemas they have to correspond to are explained in the following.

Interface

An interface can contain multiple sections (e. g. Input, InOut, Static): You can find all of these sections in the schema in the following directory:

- C:\Program Files\Siemens\Automation\Portal V*\PublicAPI\V*\Schemas\SW.InterfaceSections_v3.xsd
- C:\Program Files\Siemens\Automation\Portal V*\PublicAPI\V*\Schemas\SW.Interface.Snapshot .xsd

Compile unit

There are separate schemas for the compile units of GRAPH, LAD/FBD, STL and SCL blocks. You can find these schemas in the following directories:

- GRAPH: C:\Program Files\Siemens\Automation\Portal V*\PublicAPI\V*\Schemas\SW.PlcBlocks.Graph_v4.xsd
- LAD/FBD: C:\Program Files\Siemens\Automation\Portal V*\PublicAPI\V*\Schemas\SW.PlcBlocks.LADFBD_v3.xsd
- STL: C:\Program Files\Siemens\Automation\Portal V*\PublicAPI\V*\Schemas\SW.PlcBlocks.STL_v3.xsd
- SCL: C:\Program Files\Siemens\Automation\Portal V*\PublicAPI\V*\Schemas\SW.PlcBlocks.SCL_v2.xsd

Subschemas

There are the following additional schema definitions used by all compile units:

- Access
- Common

Access

The Access node describes for example:

- local/global members and constant usages
- FB, FC, Instruction calls
- DBs for calls

You can find the access schema in the following directory:

C:\Program Files\Siemens\Automation\Portal
V*\PublicAPI\V*\Schemas\SW.PlcBlocks.Access_v3.xsd

Common

Common contains the commonly used attributes and elements, for example different types of comments, texts and tokens.

You can find the common schema in the following directory:

C:\Program Files\Siemens\Automation\Portal V*\PublicAPI\V*\Schemas\SW.Common_v2.xsd

Note

V* refers to the installed version of TIA Portal.

2.1.3 Exporting DBs with snapshots

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project

Introduction

You can use the TIA Portal Openness to export the DB's with snapshot value in XML format to compare the values with different snapshot times. With the compare result, you can manually adapt

single start values within the TIA Portal user interface and store for a potential recovery.

The schema "SW.Interface.Snapshot.xsd" can process the exported XML-file. The Snapshot Service "InterfaceSnapshot" is provided in the namespace "Siemens.Engineering.SW.Blocks".

The file handling supports exporting DBs with snapshots:

- Export directory does not exist
- Creating of the export directory
- Export directory is readonly
- If export file already exists

The Snapshot Service is supported for Global DBs, Instance DBs and Array DB

Program code

Modify the following program code to export Snapshot Values by using the Snapshot Service:

 Copies the following program code to clipboard.

```
PlcBlock dataBlock = ...;

InterfaceSnapshot interfaceSnapshot =
dataBlock.GetService<InterfaceSnapshot>();

interfaceSnapshot.Export(new
FileInfo("C:\\temp\\MyInterfaceSnapshot.xml"),
ExportOptions.WithReadOnly);
```

Note

Export of snapshot values using the Snapshot Service is independent of the standard interface Openness export and will not influence the already existing export of the interface members. It will not be possible to import the exported XML

The snapshot values will be exported as following:

 Copies the following program code to clipboard.

```
<?xml version="1.0" encoding="utf-8"?>
<Document>
<Engineering version="V15 SP1" />
<DocumentInfo>
...
</DocumentInfo>
<SW.Blocks.InterfaceSnapshot ID="0">
<AttributeList>
<Name>GlobalDB</Name>
```

```

<Snapshot ReadOnly="true">
<SnapshotValues>
<Value Path="Static_1" Type="Bool">TRUE</Value>
<Value Path="Static_2[0]" Type="Int">1</Value>
<Value Path="Static_2[1]" Type="Int">2</Value>
<Value Path="Static_2[2]" Type="Int">3</Value>
<Value Path="Static_3" Type="DTL">DTL#1973-01-01-
00:00:00</Value>
<Value Path="Static_4.Element_1" Type="Int">7</Value>
<Value Path="Static_4.Element_2[0]" Type="Bool">FALSE</Value>
<Value Path="Static_4.Element_2[1]" Type="Bool">TRUE</Value>
<Value Path="Static_4.Element_2[2]" Type="Bool">TRUE</Value>
<Value Path="Static_4.Element_3.Element_1"
Type="Int">5</Value>
<Value Path="Static_4.Element_3.Element_2.Element_1"
Type="Bool">TRUE</Value>
<Value Path="Static_4.Element_3.Element_2.Element_2[0]"
Type="Int">100</Value>
<Value Path="Static_4.Element_3.Element_2.Element_2[1]"
Type="Int">200</Value>
</SnapshotValues></Snapshot>

<SnapshotDate ReadOnly="true">2017-12-
06T08:04:11.4590585Z</SnapshotDate>

<StructureModified ReadOnly="true">2017-12-
06T08:22:13.3292585Z</StructureModified>
</AttributeList>
</SW.Blocks.InterfaceSnapshot>
</Document>

```

If a DB does not contain any snapshot values, the content of the exported file will look like as following:

 Copies the following program code to clipboard.

```

<SnapshotValues
xlmns="http://www.siemens.com/automation/Openness/SW/Interface/Snapshot

```

```
</SnapshotValues>
```

See also

Opening a project

2.1.4 Exporting blocks with know-how protection

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)
- PLC is not online.

Introduction

The resulting XML file is similar to the export file of a block without know-how protection. With `PlcBlockProtectionProvider`, you can unlock a know-how protected block by providing the password using the Openness API and then export a KHP block with in complete with its code. A block can be imported and then be protected with a password using the API again.

If the block is exported without unlocking, only the public block interface will be exported.

- TIA Portal block -> Unlock -> Export file without protection
- Import file without protection -> Lock -> TIA Portal block

The attribute list of the block indicates that the relevant block is know-how protected.

Program code

Modify the following program code to export the visible data of a block with know-how protection to an XML file:

 Copies the following program code to clipboard.

```
private static void ExportBlock(PlcSoftware plcSoftware)
{
    PlcBlock plcBlock =
    plcSoftware.BlockGroup.Blocks.Find("MyBlock");
    plcBlock.Export(new FileInfo(string.Format(@"D:\Samples\
{0}.xml", plcBlock.Name)), ExportOptions.WithDefaults);
}
```

2.1.5 Export/Import of SCL blocks

SCL statements with export XML tags

The export operation of SCL blocks exports its equivalent XML tags based on the type of SCL statements. This operation supports the SCL networks of SCL statements in LAD/FBD blocks of SCL statements. The SCL statements are classified as text elements, operands, expressions, control, etc. The schema SW.PlcBlocks.SCL_v2.xsd is available to support you to process the SCL block with export XML tags. The SCL block statements with their corresponding exported XML tags and attributes are given below.

New line

New lines in SCL blocks are represented as NewLine XML tag.

- Contains unsigned Num attribute with default value 1.
- Num attribute does not have value 0.
- Supported only for SCL.

SCL block	XML tag
	<NewLine Num="2" />

Blank

Blank spaces in SCL blocks are represented as Blank XML tag.

- Contains unsigned Num attribute with default value 1.
- Num attribute does not have value 0.
- Supported only for SCL.
- Does not support Integer attribute available in other languages of STEP 7.

SCL block	XML tag
	<Blank Num="2"/>

Indentation of SCL block statements

In TIA Portal settings, you can modify the indentation of SCL code by accessing Options/Settings/General/Script/text editors. The following table defines the type of indentation based on the ident mode.

Ident mode	Result
None	Import operation adds the spaces as available in the source files.
Paragraph or Smart	Import operation adds the specified ident spaces in the imported file.

Based on the chosen indentation, the imported SCL block XML file are indented.

Comment

Single-line and multi-line comments in SCL blocks are represented as LineComment XML tag.

- Only LineComment tag (for single language comment) is used in SCL.
- Comment tag (for multiple language comment) is not used in SCL.
- Contains Inserted attribute with default value false
- Inserted="false" indicates "//" single comment in SCL block.
- Inserted="true" indicates "(*)" multi-line comment in SCL block.
- NoClosingBracket="true" indicates comment without closing braces in SCL block. This attribute is optional and has default value as false.
- XML does not indicate comment hierarchy in SCL block.

SCL block	XML tag
// one line comment	<pre><LineComment> <Text>one line comment</Text> </LineComment></pre>
(* one line comment second line *)	<pre><LineComment Inserted="true"> <Text>one linecomment secondline</Text> </LineComment></pre>

(* first comment (* second comment *) end first comment *)	<LineComment Inserted="true"> <Text> first comment (* second comment *) end first comment</Text> </LineComment > The nested comment is part of outer comment text.
(* comment without closing bracket	<LineComment Inserted="true" NoClosingBracket="true"> <Text> comment without closing bracket</Text> </LineComment >

Region

Regions in SCL blocks are represented as Token XML tag.

- Text XML tag represents the region_name.
- The Text attribute of the Token XML tag is case in-sensitive.
- The import operation is case in-sensitive, and the editor displays the keywords as configured in the TIA Portal settings.
- If the end_region keyword ends with ";" (semi-colon) in SCL block, the symbol ";" is placed in Text XML tag.

SCL block	XML tag
region myregion ... end_region here is the end of myregion	<Token Text="REGION" /> <Blank /> <Text>myregion</Text> <NewLine /> ... <Token Text="END_REGION" /> <Blank /> <Text>here is the end of myregion</Text> <NewLine />

end_region	<Token Text="REGION" /> <NewLine /> <LineComment .../> <Token Text="END_REGION" /> <NewLine />
region ... end_region;	<Token Text="REGION" /> <NewLine /> ... <Token Text="END_REGION" /> <Text>;</Text> <NewLine />

Pragma

Pragma in SCL blocks are represented as Token XML tag. The parameters are represented in Access XML tag with Scope attribute as LiteralConstant.

SCL block	XML tag
{PRAGMA_BEGIN 'Param1', 'Param2' (*parm 2*)} // something else {PRAGMA_END}	<Token Text="{" /> <Token Text="PRAGMA_BEGIN" /> <Blank /> <Access Scope="LiteralConstant"> <Constant> <ConstantValue>'Param1'</ConstantValue> </Constant> </Access> <Token Text="," /> <Blank /> <Access Scope="LiteralConstant"> <Constant> <ConstantValue>'Param2'</ConstantValue> </Constant> </Access> <Blank />

	<pre> <LineComment Inserted="True"> <Text>param 2</Text> </LineComment> <Token Text="," /> <Blank /> <Token Text="}" /> <NewLine /> <LineComment> <Text> something else</Text> </LineComment> <NewLine /> <Token Text="{" /> <Token Text="PRAGMA_END" /> <Token Text="}" /> </pre>
--	--

Constants: Literal constants

The constants in SCL blocks are represented by Access XML tag.

- The Scope attribute can have values like LiteralConstant, TypedConstant, LocalConstant, and GlobalConstant.
- The name of constants preceeded by "#" are ignored in XML.
- The "#" is added during the import operation of XML.
- The value of global constants represented by quotes are ignored in XML.
- The quotes are added during the import operation of XML.

Type of constant	SCL block	XML tag
Literal constant: Integer	#Out := 10;	<pre> <Access Scope="LiteralConstant"> <Constant> <ConstantValue>10</ConstantValue> <ConstantTypeInfo="true">LINT</ConstantType> </Constant> </Access> </pre>
Literal constant:		

String	#myString := 'Hello world';	<Access Scope="LiteralConstant"> <Constant> <ConstantValue>Hello world</ConstantValue> <ConstantTypeInfo="true">STRING</ConstantType> </Constant> </Access>
Literal constant: Typed	#Out := int#10;	<Access Scope="TypedConstant"> <Constant> <ConstantValue>int#10</ConstantValue> </Constant> </Access>
		Format of XML exported in ExportOptions.ReadOnly setting. <Access Scope="TypedConstant"> <Constant> <ConstantValue>int#10</ConstantValue> <StringAttribute Name="Format" Informative="true">Dec_signed</StringAttribute> <StringAttribute Name="FormatFlags" Informative="true">TypeQualifier</StringAttribute> </Constant> </Access>
Local constant	#Out := #mylocal;	<Access Scope="LocalConstant"> <Constant Name="mylocal" /> </Access>

Local constant	#Out := #mylocal;	Format of XML exported in ExportOptions.ReadOnly setting. <pre> <Access Scope="LocalConstant"> <Constant Name="mylocal"> <ConstantType Informative="true">Int</ConstantType> <ConstantValue Informative="true">10</ConstantValue> <StringAttribute Name="Format" Informative="true">Dec_signed</StringAttribute> </Constant> </Access> </pre>
Global constant	#Out := "myglobal";	<pre> <Access Scope="GlobalConstant"> <Constant Name="myglobal" /> </Access> </pre>
		Format of XML exported in ExportOptions.ReadOnly setting. <pre> <Access Scope="GlobalConstant"> <Constant Name="myglobal"> <ConstantType Informative="true">Int</ConstantType> <ConstantValue Informative="true">10</ConstantValue> <StringAttribute Name="Format" Informative="true">Dec_signed</StringAttribute> </Constant> </Access> </pre>

The address constants are not supported in SCL blocks, and it is ignored in this table.

Variables

The local and global variables in SCL blocks are represented by Access XML tag.

- The Scope attribute has values of LocalVariable and GlobalVariable
- The XML tags for assigning the value 10 is ignored here.

Type of variable	SCL block	XML tag
Local variable	#Out := 10;	<Access Scope="LocalVariable"> <Symbol> <Component Name="Out" /> </Symbol> </Access>
Global variable	"Tag_3" := 10;	<Access Scope="GlobalVariable"> <Symbol> <Component Name="Tag_3" /> </Symbol> </Access>
		Format of XML exported in ExportOptions.ReadOnly setting. <Access Scope="GlobalVariable"> <Symbol> <Component Name="Tag_3" /> <Address Area="Memory" Type="Int" BitOffset="96" Informative="true" /> </Symbol> </Access>

Expressions

The simple expressions in SCL blocks are represented by Access XML tag. The Scope attribute has value of LocalVariable for the expressions

SCL block	XML tag
#a := #b + #c;	<Access Scope="LocalVariable"> <Symbol> <Component Name="a" /> </Symbol> </Access> <Blank />

	<pre> <Token text=":=" /> <Blank /> <Access Scope="LocalVariable"> <Symbol> <Component Name="b" /> </Symbol> </Access> <Blank /> <Token text="+" /> <Blank /> <Access Scope="LocalVariable"> <Symbol> <Component Name="c" /> </Symbol> </Access> <Token text=";" /> </pre>
--	--

Control structures in SCL blocks

The control statements like IF, CASE, FOR, WHILE, REPEAT, GOTO, EXIT, CONTINUE, and RETURN are represented by Token XML tag.

- The conditional symbols used in SCL block such as >, <, & are represented as escape sequences (< > &) in XML.
- These combination of XML tags are applicable only for SCL blocks. An exception is thrown for other languages.

		<Access Scope="LocalVariable"> <Symbol> <Component Name="c" /> </Symbol> </Access> <Blank /> <Token Text="THEN" /> <NewLine /> <Blank Num="4" /> <Token Text=";" /> <NewLine /> <Token Text="END_IF" /> <Token Text=";" />
CASE	CASE #a OF 1 (*test*): // Statement section case 1 ; 2..4: // Statement section case 2 to 4 ; ELSE // Statement section ELSE ; END_CASE;	<Token Text="CASE" /><Blank /> <Access Scope="LocalVariable"> <Symbol> <Component Name="a" /> </Symbol> </Access> <Blank /> <Token Text="OF" /> <NewLine /> <Blank Num="2"/> <Access Scope="LiteralConstant"> <Constant> <ConstantValue>1</ConstantValue> <ConstantType Informative="true">LINT</ConstantType> </Constant> </Access> <Blank /> <LineComment Inserted="true"> <Text>test</Text> </LineComment >

	<pre> <Token Text=":" /> <Blank /> <LineComment> <Text> Statement section case 1</Text> </LineComment > <NewLine /> <Blank Num="4"/> <Token Text=";" /> <NewLine /> <Blank Num="2"/> <Access Scope="LiteralConstant"> <Constant> <ConstantValue>2</ConstantValue> <ConstantType Informative="true">LINT</ConstantType> </Constant> </Access> <Token Text=".." /> <Blank Num="2"/> <Access Scope="LiteralConstant"> <Constant> <ConstantValue>4</ConstantValue> <ConstantType Informative="true">LINT</ConstantType> </Constant> </Access> <Blank /> <LineComment> <Text> Statement section case 2 to 4</Text> </LineComment > <NewLine /> <Blank Num="4"/> <Token Text=";" /> <NewLine /> <Blank Num="2"/> <Token Text="ELSE" /> </pre>
--	---

		<pre><NewLine /> <Blank Num="4"/> <Token Text=";" /> <NewLine /> <Token Text="END_CASE" /> <Token Text=";" /></pre>
--	--	---

		<pre> <Text> Statement section FOR</Text> </LineComment > <NewLine /> <Blank Num="2" /> <Token Text=";" /> <NewLine /> <Token Text="END_FOR" /> <Token Text=";" /> </pre>
--	--	---

WHILE	<pre> WHILE #a<#b DO // Statement section WHILE ; END_WHILE; </pre>	<pre> <Token Text="WHILE" /> <Blank /> <Access Scope="LocalVariable"> <Symbol> <Component Name="a" /> </Symbol> </Access> <Token Text="<" /> <Access Scope="LocalVariable"> <Symbol> <Component Name="b" /> </Symbol> </Access> <Blank /> <Token Text="DO" /> <NewLine /> <Blank Num="2" /> <LineComment> <Text> Statement section WHILE</Text> </LineComment > <NewLine /> <Blank Num="2" /> <Token Text=";" /> <NewLine /> <Token Text="END_WHILE" /> <Token Text=";" /> </pre>
-------	--	--

		<pre> <Token Text=";" /> <NewLine /> <Token Text="UNTIL" /> <Blank /> <Access Scope="LocalVariable"> <Symbol> <Component Name="a" /> </Symbol> </Access> <Token Text="<" /> <Access Scope="LocalVariable"> <Symbol> <Component Name="b" /> </Symbol> </Access> <Blank /> <Token Text="END_REPEAT" /> <Token Text=";" /> </pre>
--	--	---

		</LineComment>
	GOTO (*comment*) here;	XML example for GOTO label usage <Token Text="GOTO" /> <Blank /> <LineComment inserted="true"> <Text>comment</Text> </LineComment> <Blank /> <Access Scope="Label"> <Label Name="here" /> </Access> <Token Text=";" />

Referencing attributes

The SCL block referencing attributes are represented by AccessModifier attribute of Component tag.

- For simple referencing, AccessModifier has value as Reference.
- For array referencing, AccessModifier has value as ReferenceToArray.

SCL block	XML tag
RefToUDT^(*RefToUDT*).element	<Symbol> <Component Name="RefToUDT" AccessModifier="Reference" /> <Token Text="^" /> <LineComment Inserted="True"> <Text>RefToUDT</Text> </LineComment> <Token Text="." /> <Component Name="element" /> </Symbol>
RefToArrayOfUDT^(*RefToArrayOfUDT*) [#i].element	<Symbol> <Component Name="RefToArrayOfUDT" AccessModifier="ReferenceToArray" />

	<pre> <Token Text="^" /> <LineComment Inserted="True"> <Text>RefToArrayOfUDT</Text> </LineComment> <Token Text="[" /> <Access Scope=LocalVariable> <Symbol> <Component Name="i" /> </Symbol> </Access> <Token Text="]" /> </Component> <Token Text="." /> <Component Name="element" /> </Symbol> </pre>
--	---

2.1.6 Export/Import of structured types of SCL blocks

SCL structured types with export XML tags

In the SCL structured types, you can add blanks, new lines, and comments in the SCL statements. The SCL structured statements with its corresponding exported XML tags and attributes are given below.

Global access

In SCL statements, the global access variables and constants are represented in quotes. The comments written between the variables and address parts are represented by LineComment XML tag.

	<pre> </LineComment> <Component Name="Static_1" /> <LineComment Inserted="True"> <Text>comment 2</Text> </LineComment> <Token Text="." /> <Component Name="Static_2" /> </Symbol> </Access> </pre>
<pre> "Data_block_1".Static_1 := 10 </pre>	Format of XML exported in ExportOptions.None setting. <pre> <Access Scope="GlobalVariable"> <Symbol> <Component Name="Data_block_1" /> <Token Text="." /> <Component Name="Static_1" /> </Symbol> </Access> </pre>
	Format of XML exported in ExportOptions.ReadOnly setting. <pre> <Access Scope="GlobalVariable"> <Symbol> <Component Name="Data_block_1" /> <Token Text="." /> <Component Name="Static_1" /> <Address Area="DB" Type="Word" BlockNumber="1" BitOffset="0" Informative="true" /> </Symbol> </Access> </pre>

Usage of Quotes and

The quotes used in the first level describes the type of variable, and used to escape special characters in SCL statements. When quotes are used in first level, it defines the variable as global variable. If the quotes are used after #, they represent the escape sequence of special characters like #, and spaces.

- To represent the differential usage, XML file uses BooleanAttributes tag with Name attribute. The Name contain values such as HasQuotes and HasHash.
- To define structure in scope attribute, # is defined.
- These values are applicable only for SCL.
- The default values for these tags were FALSE, but the values never gets exported in ExportOptions.WithDefaults settings too.

SCL block	XML tag
"a".#b."c".#"d"	<pre> <Access Scope="GlobalVariable"> <Symbol> <Component Name="a" /> <Token Text="." /> <Component Name="b"> <BooleanAttribute Name="HasHash">TRUE</BooleanAttribute> </Component> <Token Text="." /> <Component Name="c"> <BooleanAttribute Name="HasQuotes">TRUE</BooleanAttribute> </Component> <Token Text="." /> <Component Name="d"> <BooleanAttribute Name="HasQuotes">TRUE</BooleanAttribute> <BooleanAttribute Name="HasHash">TRUE</BooleanAttribute> </Component> </Symbol> </Access /> </pre>

Array

SCL allows to add comment within the array indexes around "[" and "]". To mark the existence of array, XML file uses AccessModifier attribute in Component tag.

- If Accessmodifier contains the value Array, then a child tag Access is mandatoy to indicate the index variable of the array.

- The default value for AccessModifier is None.

SCL block	XML tag
#a.b[#i+#j,#k+#l].c	<pre> <Access Scope="LocalVariable"> <Symbol> <Component Name="a" /> <Token Text="." /> <Component Name="b" AccessModifier="Array" /> <Token Text="[" /> <Access Scope=LocalVariable> <Symbol> <Component Name="i" /> </Symbol> </Access> <Token Text="+" /> <Access Scope=LocalVariable> <Symbol> <Component Name="j" /> </Symbol> </Access> <Token Text="," /> <Access Scope=LocalVariable> <Symbol> <Component Name="k" /> </Symbol> </Access> <Token Text="+" /> <Access Scope=LocalVariable> <Symbol> <Component Name="l" /> </Symbol> </Access> <Token Text="]" /> </Component> </pre>

	<pre> <Token Text="." /> <Component Name="c" /> </Symbol> </Access> </pre>
--	--

Absolute Access

SCL allows different types of access such as absolute, absolute offset, mixed (database and member variable), slice, peripheral, and direct type. The absolute access specifiers are represented by Address tag in XML.

- The % character of the DB is not written in XML. It is created automatically during the import.
- Blanks are allowed between the address parts

SCL Block	XML Tag
%DB20 . DBW10	<pre> <Access Scope="Address"> <Symbol> <Address Area="DB" BlockNumber="20" /> <Blank /> <Token Text="." /> <Blank /> <Address Area="DB" BitOffset="80" Type="Word"/> </Symbol> </Access> </pre>
%DB20.DBX10.3 := true;	The following XML is valid for all languages except SCL. <pre> <Access Scope="Address"> <Address Area="DB" BlockNumber="20" BitOffset="83" Type="Bool" /> </Access> </pre> The following XML is valid for SCL. <pre> <Access Scope="Address"> <Symbol> </pre>

	<pre> <Address Area="DB" BlockNumber="20" /> <Token Text="." /> <Address Area="DB" BitOffset="83" Type="Bool"/> </Symbol> </Access> </pre>
--	--

Absolute offset

In STL, AbsoluteOffset tag represents the absolute offset access. In SCL, Address tag is used for absolute access.

SCL Block	XML Tag
#Input_DB_ANY.%DBX2.3 := TRUE;	<pre> <Access Scope="LocalVariable"> <Symbol> <Component Name="Input_DB_ANY" /> <Token Name="." /> <Address BitOffset="19" Type="Bool" /> </Symbol> </Access> </pre>

Slicing

In SCL, the SliceAccessModifier attribute is not supported and the slicing is represented by Token tag.

	<pre> <Component Name="member"/> <LineComment Inserted="True"> <Text>3</Text> </LineComment> <Token Text="." /> <LineComment Inserted="True"> <Text>4</Text> </LineComment> <Token Text="%x1" /> </Symbol> </Access> </pre>
--	---

Peripheral access

The peripheral access is represented by Token tag.

SCL Block	XML Tag
"tag_1"(*1*).(*2*)member:P	<pre> <Access Scope="GlobalVariable"> <Symbol> <Component Name="tag_1" /> <LineComment Inserted="True"> <Text>1</Text> </LineComment> <Token Text="." /> <LineComment Inserted="True"> <Text>2</Text> </LineComment> <Component Name="member"/> <Token Text=":P" /> </Symbol> </Access> </pre>

Direct type access

The TypeOf and TypeOfDB instructions are handled either with system type or user defined type. The types are represented in Access tag with Scope attribute containing values of SystemType and UserType.

SCL Block	XML Tag
Example for system type if TypeOf(#inVariant) = TO_SpeedAxis then ... end_if	<pre> <Token text="=" /> </Blank> <Access Scope="SystemType"> <DataType>TO_SpeedAxis</DataType> </Access> </pre>
Example for user defined type if TypeOf(#inVariant) = "aUserDefinedType" then ... end_if	<pre> <Token text="=" /> </Blank> <Access Scope="UserType"> <DataType>aUserDefinedType</DataType> </Access> </pre>

2.1.7 Export/Import of SCL call blocks

SCL call blocks with export XML tags

SCL call paramaters are respresented by Parameter tag in XML. The informative attribute is used to represent the non-assigned FB parameters and return values such as timestamp, flag information, etc. The XML format follows the same arbitrary order followed in the SCL block.

An example for block call is given below.

SCL block	XML tag
#Callee_Instance(Input_1 := 5);	<pre> Format of XML exported in ExportOptions.None setting <Access Scope="Call"> <CallInfo BlockType="FB"> <Instance Scope="LocalVariable"> <Component Name="Callee_Instance" /> </Instance> <Token text="(" /> <Parameter Name="Input_1"> </pre>

```

<Blank />
<Token text=":" />
<Blank />
<Access Scope="LiteralConstant">
  <Constant>
    <ConstantType>Int</ConstantType>
    <ConstantValue>5</ConstantValue>
  </Constant>
</Access>
</Parameter>
<Token text=")" />
  </CallInfo>
</Access>
<Token text=";" />

```

Format of XML exported in ExportOptions.ReadOnly setting

```

<Access Scope="Call">
  <CallInfo BlockType="FB">
    <IntegerAttribute Name="BlockNumber" Informative="true">1</IntegerAttribute>
    <DateAttribute Name="ParameterModifiedTS" Informative="true">2016-10-
24T08:27:34</DateAttribute>
    <Instance Scope="LocalVariable">
      <Component Name="Callee_Instance" />
    </Instance>
  <Token text="(" />
    <Parameter Name="Input_1">
<StringAttribute Name="InterfaceFlags"
Informative="true">S7_Visible</StringAttribute>
    <Blank />
    <Token text=":" />
    <Blank />
    <Access Scope="LiteralConstant">
      <Constant>
        <ConstantType>Int</ConstantType>
        <ConstantValue>5</ConstantValue>

```

```
</Constant>  
</Access>  
</Parameter>  
<Token text=")" />  
  </CallInfo>  
</Access>  
<Token text=";" />
```

Unconnected parameters example

The FB has 4 parameters where a, b, c, and d. b and d are not connected.

	<pre> <ConstantValue>True</ConstantValue> </Constant> </Access> </Parameter> <Parameter Name="d" Informative="true"/> <Token text=")" /> </CallInfo> </Access> </pre>
--	---

One parameter example

SCL block allows you to omit the parameter name. This parameter is represented as NamelessParameter tag. The NamelessParameter tag has no attributes and it is applicable only for SCL.

SCL block	XML tag
"Block_4_DB"(TRUE);	<pre> <Access Scope="Call"> <CallInfo Name="Block_4" BlockType="FB"> <Instance Scope="GlobalVariable"> <Component Name="Block_4_DB" /> </Instance> <Token text="(" /> <NamelessParameter> <Access Scope="LiteralConstant"> <Constant> <ConstantType>Bool</ConstantType> <ConstantValue>TRUE</ConstantValue> </Constant> </Access> </NamelessParameter> <Token text=")" /> </CallInfo> </Access> </pre>

Expression as actual parameter

SCL block	XML tag
#Callee_Instance(Input_1 := #a+3);	<pre> <Access Scope="Call"> <CallInfo BlockType="FB"> <Instance Scope="LocalVariable"> <Component Name="Callee_Instance" /> </Instance> <Token text="(" /> <Parameter Name="Input_1"> <Blank /> <Token text=":=" /> <Blank /> <Access Scope="LocalVariable"> <Symbol> <Component Name="a" /> </Symbol> </Access> <Token text="+" /> <Access Scope="LiteralConstant"> <Constant> <ConstantType>Int</ConstantType> <ConstantValue>3</ConstantValue> </Constant> </Access> </Parameter> <Token text=")" /> </CallInfo> </Access> <Token text=";" /> </pre>

Expression as actual parameter without formal paramter

SCL block	XML tag
-----------	---------

#Callee_Instance(#a+3);	<Access Scope="Call"> <CallInfo BlockType="FB"> <Instance Scope="LocalVariable"> <Component Name="Callee_Instance" /> </Instance> <Token text="(" /> <NamelessParameter> <Access Scope="LocalVariable"> <Symbol> <Component Name="a" /> </Symbol> </Access> <Token text="+" /> <Access Scope="LiteralConstant"> <Constant> <ConstantType>Int</ConstantType> <ConstantValue>3</ConstantValue> </Constant> </Access> </NamelessParameter> <Token text=")" /> </CallInfo> </Access> <Token text=";" />
-------------------------	--

Function call

	<pre> <Token text=":=" /> <Blank /> <Access Scope="Call"> <CallInfo Name="MyFunction" BlockType="FC"> <Token text="(" /> <Parameter Name="Param_1"> ... </pre>
--	--

Absolute call

In SCL, the call can be initiated using absolute address of the DB. Due to absolute address, the Name attribute of CallInfo node is empty.

A recoverable exception is thrown by the import of

- An "Address" node is available, with valid value of Name attribute.
- Non-existence of Address node with no valid value of Name attribute.

SCL block	XML tag
%DB20(...);	<pre> <Access Scope="Call"> <CallInfo Name="" BlockType="FB"> <Instance Scope="GlobalVariable"> <Address Area="DB" BlockNumber="20" /> </Instance> <Token text="(" /> <Parameter> ... </Parameter> <Token text=")" /> </CallInfo> </Access> </pre>

Instruction

The instruction in SCL block is checked in system library during the import operation, and the instruction versions are not exported in export operation.

The general instruction type is given below.

SCL block	XML tag
#myInt := ATTACH(OB_NR := 1, EVENT := 15, ADD := TRUE);	Format of XML exported in ExportOptions.ReadOnly setting <pre> <Access Scope="LocalVariable"> <Symbol> <Component Name="myInt" /> </Symbol> </Access> <Blank /> <Token text=":=" /> <Blank /> <Access Scope="Call"> <Instruction Name="ATTACH"> <Token text="(" /> <Parameter Name="OB_NR"> <Blank /> <Token text=":=" /> <Blank /> <Access Scope="LiteralConstant"> <Constant> <ConstantType>OB_ATT</ConstantType> <ConstantValue>1</ConstantValue> </Constant> </Access> </Parameter> <Token text="," /> <Blank /> <Parameter Name="EVENT"> <Blank /> <Token text=":=" /> <Blank /> <Access Scope="LiteralConstant"> <Constant> </pre>

	<pre> <ConstantType>EVENT_ATT</ConstantType> <ConstantValue>15</ConstantValue> </Constant> </Access> </Parameter> <Token text="," /> <Blank /> <Parameter Name="ADD"> <Blank /> <Token text=":=" /> <Blank /> <Access Scope="LiteralConstant"> <Constant> <ConstantType>Bool</ConstantType> <ConstantValue>TRUE</ConstantValue> </Constant> </Access> </Parameter> <Parameter Name="RET_VAL" Informative="true" /> <Token text=")" /> </Instruction> </Access> <Token text=";" /> </pre>
--	--

Instruction with template

When the template parameter is complements the instruction name, the export of the template parameter is necessary. If a "TemplateValue" tag with attribute Type="Type" follows the Instruction tag, the import operation concatenates the template value to the instruction name.

SCL block	XML tag
<pre> "tag_4" := MIN_DINT(IN1:="Tag_1", IN2:="Tag_2", IN3:="Tag_3"); </pre>	<pre> <Access Scope="GlobalVariable"> <Symbol> <Component Name="Tag_4" /> </Symbol> </pre>

```

</Access>
...
<Access Scope="Call">
  <Instruction Name="MIN">
    <TemplateValue Name="value_type" Type="Type">DInt</TemplateValue>
    ...
  <Parameter Name="IN1">
    ...
    <Access Scope="GlobalVariable">
      <Symbol>
        <Component Name="Tag_1" />
      </Symbol>
    </Access>
  </Parameter>
  ...
  <Parameter Name="IN2">...
    <Access Scope="GlobalVariable">
      <Symbol>
        <Component Name="Tag_2" />
      </Symbol>
    </Access>
  </Parameter>
  ...
  <Parameter Name="IN3">
    ...
    <Access Scope="GlobalVariable">
      <Symbol>
        <Component Name="Tag_3" />
      </Symbol>
    </Access>
  </Parameter>
  ...
  </Instruction>
</Access>

```

	...
--	-----

Conversion

For conversion functions, the real instruction name and its template values are not exported. Instead, the name used in the SCL block is exported.

SCL block	XML tag
#output_1 := TIME_TO_S5TIME(#input_1);	<Access Scope="LocalVariable"> <Symbol> <Component Name="output_1" /> </Symbol> </Access> ... <Access Scope="Call"> <Instruction Name="TIME_TO_S5TIME"> <Token text="(" /> <NamelessParameter> <Access Scope="LocalVariable"> <Symbol> <Component Name="input_1" /> </Symbol> </Access> </NamelessParameter> <Token text=")" /> </Instruction> </Access> ...

Instruction with instance

The instance and instruction are separated by blanks. Blanks are optional, and they can be represented by new lines and comments. The instruction TON is represented by Name attribute of Instruction tag.

SCL block	XML tag
-----------	---------

IEC_Timer_0_DB . TON (IN:="Tag_1", PT:="Tag_2");	<Access Scope="GlobalAccess"> <Symbol> <Component Name="IEC_Timer_0_DB" /> </Symbol > </Access> <Blank /> <Token text="." /> <Blank /> <Access Scope="Call"> <Instruction Name="TON"> <Blank /> <Token text="(" /> <Parameter Name="IN"> <Access Scope="GlobalVariable"> <Symbol> <Component Name="Tag_1" /> </Symbol> </Access> </Parameter> ... <Token text=")" /> </Instruction> </Access> <Token text=";" />
---	--

Alarm constant

The alarm constants are only used in S7 400 PLCs, and the exported XML is similar to other languages.

	<pre> <Instance Scope="GlobalVariable"> <Component Name="Block_1_DB" /> </Instance> <NamelessParameter> <Access Scope="AlarmConstant"> <Constant> <ConstantType>C_Alarm_8</ConstantType> <ConstantValue>16#0000_0001</ConstantValue> </Constant> </Access> </NamelessParameter > </CallInfo> </Access> </pre>
	Format of XML exported in ExportOptions.ReadOnly setting <pre> <Access Scope="Call"> <CallInfo Name="Block_1" BlockType="FB"> <Instance Scope="GlobalVariable"> <Component Name="Block_1_DB" /> </Instance> <NamelessParameter> <Access Scope="AlarmConstant" > <Constant> <ConstantValue>16#00000001</ConstantValue> <ConstantType>C_Alarm</ConstantType> <StringAttribute Name="Format" Informative="true">Hex</StringAttribute> </Constant> </Access> </NamelessParameter> </CallInfo> </Access> </pre>

ENO (Enable Output)

To support the ENO construct in SCL block, an attribute named "Scope" with value "PredefinedVariable" is used in the "Access" tag. It also contains the "PredefinedVariable" tag as

child of the Access tag.

- The "PredefinedVariable" tag has one mandatory "Name" attribute.
- The scope "PredefinedVariable" and the tag "PredefinedVariable" are only allowed for SCL.

SCL block	XML tag
Call(..., ENO => ENO);	<pre><Access Scope="Call"> <CallInfo BlockType="FC"> <Token text="(" /> ... <Token text="," /> <Blank /> <Parameter Name="ENO"> <Blank /> <Token text="=>" /> <Blank /> <Access Scope="PredefinedVariable"> <PredefinedVariable Name="ENO" /> </Access> </Parameter> <Token text=")" /> </CallInfo> </Access> <Token text=";" /></pre>

	</Symbol> </Access> <Blank /> <Token Text="THEN" />
--	--

2.1.8 Export/Import multilingual comments in SCL

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open
See Opening a project

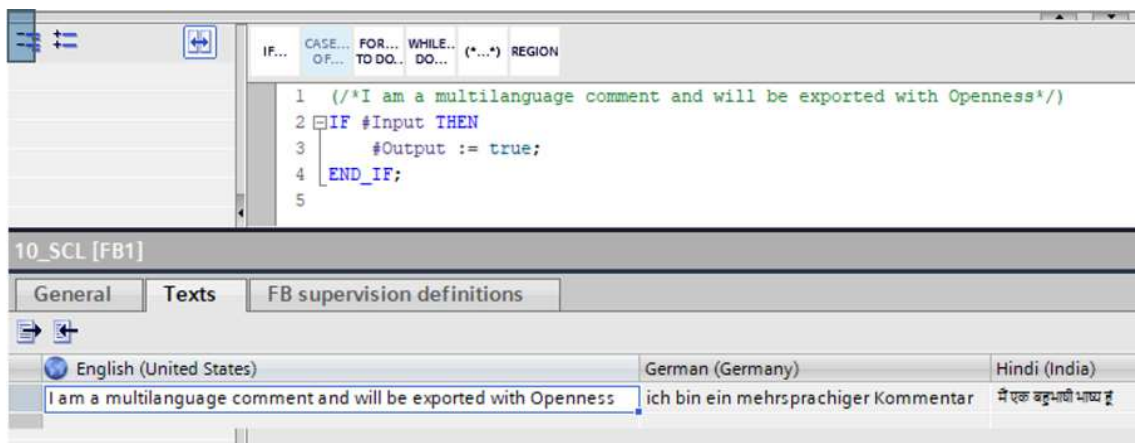
Application

It is possible in TIA Portal Openness to support import and export multilingual comments in SCL editor by the following ways:

- During SCL export in Openness, all the multilingual comments with its translation, according to enabled project languages, are exported into the same openness xml file.
- During SCL import in Openness, all the multilingual comments with its translation, according to enabled project languages, shall be imported back from the openness xml file. It shall be reflected in the project text area. The language gets enabled in if the language is not enabled in the project during import.
- After import, it shall be verified that all multilingual comment in SCL editor is linked to the appropriate comments in the project text.

Example

The following figure shows an example of Multilingual comment in SCL Editor:



The following figure shows an example for export of multilingual comment in openness XML file:

```
<Comment Inserted="true" UID="21">
  <MultilanguageText Lang="en-US">I am a multilanguage comment and will be exported with Openness</MultilanguageText>
  <MultilanguageText Lang="de-DE">ich bin ein mehrsprachiger Kommentar</MultilanguageText>
  <MultilanguageText Lang="hi-IN">मैं एक बहुभाषी भाष्य हूँ</MultilanguageText>
</Comment>
```

See also

Connecting to the TIA Portal
Opening a project

2.1.9 Exporting failsafe blocks

Exporting failsafe blocks

It is now possible to import and export the Failsafe blocks, but F-system blocks cannot be exported.

Importing failsafe blocks

Consistent F-blocks can be exported and re-imported. They will be created as F-blocks.

It will be imported as standard block if you remove the prefixes "F_" from the value of all attributes "ProgrammingLanguage".

See also

Connecting to the TIA Portal
Opening a project
Exporting blocks
Exporting blocks with know-how protection

2.1.10 Exporting system blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project
- The project contains a system block.
- The system block is not a F-block
- PLC is not online.

Introduction

Only visible system blocks are available in the blocks composition, for example no FB or FC blocks are available in the block composition. The resulting XML file is similar to the export file of a block.

Program code

Modify the following program code to export the visible data of a block to an XML file:

 Copies the following program code to clipboard.

```
//Exports system blocks

private static void ExportSystemBlocks(PlcSoftware
plcsoftware)
{
    PlcSystemBlockGroup sbSystemGroup =
    plcsoftware.BlockGroup.SystemBlockGroups[0];

    foreach (PlcSystemBlockGroup group in sbSystemGroup.Groups)
    {
        foreach (PlcBlock block in group.Blocks)
        {
            block.Export(new FileInfo(string.Format(@"D:\Samples\{ 0
}.xml", block.Name)), ExportOptions.WithDefaults);
        }
    }
}
```

2.1.11 Exporting GRAPH blocks with multi-language text

XML structure of GRAPH blocks with multi-language text

The export XML for GRAPH blocks contains the translated step names and transition names of the GRAPH. These translated multi-language text are represented as StepName and TransitionName elements under the parent element Step and Transition respectively. These elements contain one MultiLanguageText element for each supported language. The texts for the languages which are not set explicitly are not exported. If no translation is made, the StepName and TransitionName elements are not exported. The StepName and TransitionName elements are optional. The TIA Portal Openness XML import operation throws a recoverable exception for the graph versions < V5.0.

Example for StepName element

 Copies the following program code to clipboard.

```

<Steps>

  <Step Number="1" Init="true" Name="Step1"
MaximumStepTime="T#10S" WarningTime="T#7S">

    <StepName>

      <MultiLanguageText Lang="de-
DE">stepDE</MultiLanguageText>

      <MultiLanguageText Lang="en-
US">stepEN</MultiLanguageText>

      <MultiLanguageText Lang="it-
CH">stepIT</MultiLanguageText>

    </StepName>

    ..

  </Step>

  ..

</Steps>

```

Example for TransitionName element

 Copies the following program code to clipboard.

```

<Transitions>

  <Transition IsMissing="false" Name="Trans1" Number="1"
ProgrammingLanguage="LAD">

    <TransitionName>

      <MultiLanguageText Lang="de-
DE">transDE</MultiLanguageText>

      <MultiLanguageText Lang="en-
US">transEN</MultiLanguageText>

      <MultiLanguageText Lang="it-
CH">transIT</MultiLanguageText>

    </TransitionName>

    ..

  </Transition>

  ..

</Transitions>

```

2.1.12 Importing block

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project
- PLC is not online.

Introduction

The TIA Portal Openness API supports the import of blocks with "LAD", "FBD", "GRAPH", "SCL" or "STL" programming languages from an XML file. The following block types are supported:

- Function blocks (FB)
- Functions (FC)
- Organization blocks (OB)
- Global data blocks (DB)

Note

Importing optimized data blocks

Optimized data blocks are only supported by CPUs as of S7-1200. If you import optimized data blocks into S7-300 or S7-400, an exception is thrown and the import fails.

Response to importing

The following rules apply when importing a block:

- The XML file can contain less data than the block in the project, e.g., fewer parameters.
- Redundant information, such as call information, must be identical in the project and in the XML file. Otherwise, an exception is thrown.
- The data in the XML file may be "inconsistent" regarding their ability to be compiled in the TIA Portal.
- Attributes with the attributes "ReadOnly=True" and "Informative=True" are not imported.
- Missing instance DBs are not created automatically.
- If no block number is specified in the xml file, the block number is assigned automatically.
- If the block is not existing in the project and no version information is specified in the xml file, the version "0.1" is assigned.

Program code

Modify the following program code:

 Copies the following program code to clipboard.

```
//Import blocks

private static void ImportBlocks(PlcSoftware plcSoftware)
{
    PlcBlockGroup blockGroup = plcSoftware.BlockGroup;

    IList<PlcBlock> blocks = blockGroup.Blocks.Import(new
    FileInfo(@"D:\Blocks\myBlock.xml"), ImportOptions.Override);
}
```

Modify the following program code:

 Copies the following program code to clipboard.

```
//Import system blocks

private static void ImportSystemBlocks(PlcSoftware
plcSoftware)
{
    PlcBlockSystemGroup systemblockGroup =
    plcSoftware.BlockGroup.SystemBlockGroups[0].Groups[0];

    IList<PlcBlock> blocks = systemblockGroup.Blocks.Import(new
    FileInfo(@"D:\Blocks\myBlock.xml"), ImportOptions.Override);
}
```

2.1.13 Exporting blocks

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project
- PLC is not online.

Introduction

The API interface supports exporting consistent blocks and user data types to an XML file.

The XML file receives the name of the block. The following block types are supported:

- Function blocks (FB)
- Functions (FC)
- Organization blocks (OB)
- Global data blocks (DB)

The following programming languages are supported:

- STL
- FBD
- LAD
- GRAPH
- SCL

Attributes applicable for all blocks

The following attributes are exported in all blocks with the selected `ExportOptions` (see Exporting configuration data). Attributes in bold typeface are always exported.

Additional information is available in the TIA Portal information system under "Overview of block attributes".

Attribute	Type	Default value	ReadOnly
AutoNumber	Bool	true	false
CodeModifiedDate	DateTime	-	true
CompileDate	DateTime	-	true
CreationDate	DateTime	-	true
HeaderAuthor	String	""	false
HeaderFamily	String	""	false
HeaderName	String	""	false
HeaderVersion	String	"0.1"	false
Interface	String	empty interface	false

InterfaceModifiedDate	DateTime	-	true
IsConsistent	Bool	-	true
IsKnowHowProtected ¹	Bool	false	true
IsWriteProtected	Bool	false	true
MemoryLayout	enum MemoryLayout	-	false
ModifiedDate	DateTime	-	true
Name	String	-	false
Number	Int32	next available number	false
ParameterModified	DateTime	-	true
PLCSimAdvancedSupport	Bool	false	true
ProgrammingLanguage	enum ProgrammingLanguage	-	false
StructureModified	DateTime	-	true
¹ The IsKnowHowProtectedattribute is applicable for UDT too.			

Note

There are certain conditions where MemoryLayout attribute can be readonly.

Dynamic accessible general attribributes

The following attributes which can be read only in some conditions:

Attribute	ReadOnly condition
-----------	--------------------

AutoNumber	All KnowHowProtected blocks, All Classic OBs; Plus OBs: DiagnosticErrorInterrupt, IOAccessError, ProgrammingError, PullOrPlugOfModules, RackOrStationFailure, Status, TimeErrorInterrupt, Update
HeaderVersion	System- and KnowHowProtected DBs; ArrayDBs that originated from system library
HeaderName	
HeaderFamily	
HeaderAuthor	
MemoryLayout	Classic blocks, System Know how protected blocks, ArrayDBs, IDBofFBs, Graph blocks

Attributes applicable for ArrayDB block

The following attributes are exported for ArrayDB block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
ArrayDataType	String	-	true
ArrayLimitUpperBound	Int32	-	true

Attributes applicable for DB block

The following attributes are exported in DB block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
IsOnlyStoredInLoadMemory	Bool	false	false
IsPLCDB	Bool	false	false
IsWriteProtectedInAS	Bool	false	false

Attributes applicable for FB block

The following attributes are exported for FB blocks with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
AssignedProDiagFB	String	-	-
ISMultInstanceCapable	Bool	-	true
Supervisions	String	no supervisions	true for IDB of FB and false for FB

Attributes applicable for DB and FB blocks

The following attributes are exported in DB and FB block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
IsIECCheckEnabled	Bool	false	false
IsRetainMemResEnabled ¹	Bool	false	false
MemoryReserve	Unsigned	0	false
RetainMemoryReserve ²	Unsigned	0	false
² If the "IsRetainMemResEnabled" attribute's value is "false", and the "RetainMemoryReserve" attribute is not equal to "0", an exception is thrown.			

Attributes applicable for FB, DB and IDB blocks

The following attributes are exported in FB, DB, and IDB blocks with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
DownloadWithoutReinit	Bool	false	true

Attributes applicable for FB and FC blocks

The following attributes are exported for FB and FC block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
-----------	------	---------------	----------

LibraryType	String	-	true
LibraryTypeVersionGuid	String	-	true
¹ LibraryTypeGuid	String	-	true

Note

¹ In TIA Portal Openness v20, The LibraryTypeGuid is added as a new read-only attribute to the attribute list of a PLC block / PLC data type in SimaticML

You are not able to export the above attributes for CodeBlocks or PlcTypes when these objects are not instances of a library user type.

You can ignore these attributes during import. The import process will produce a non-type-connected CodeBlock or PlcType even if these attributes are present in SimaticML.

Attributes applicable for FB and FC (STL) blocks

The following attributes are exported for FB and FC (STL) block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
ParameterPassing	Bool	false	false

Attributes applicable for FB, FC and instance DB of an FB block

The following attributes are exported for FB, FC and instance DB of an FB block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
UDABlockProperties	String	""	false
UDAEnableTagReadback	Bool	false	false

Attributes applicable for instance DB of FB and UDT

The following attributes are exported for instance DB of FB and UDT block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
InstanceOfName	String	""	false
InstanceOfNumber	Unsigned Short	-	true
InstanceOfType	enum BlockType	-	true
OfSystemLibElement	String	""	false
OfSystemLibVersion	String	""	false

Attributes applicable for OB block

The following attributes are exported in OB block for specific Plus PLCs with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
ApplicationCycle	Single	-	true
AutomaticMinimum	Bool	-	true
ConstantName	String	-	true
CycleTimeDistributedIO	Single	-	true
CyclicApplicationCycleTime	Single	-	true
CyclicTime	Int32	100000	true
DataExchangeMode	OBDDataExchangeMode	Cyclic	true
DelayTime	Double	-	true
DistributedIOName	String	-	true
DistributedIONumber	Int32	-	true
EnableTimeError	Bool	-	true

EventClass	String	-	true
EventsToBeQueued	Int32	-	true
EventThresholdForTimeError	Int32	-	true
Execution	OBExecution	Never	true
Factor	Single	-	true
PhaseOffset	Int32	0	true
PriorityNumber	Int32	-	true
ProcessImagePartNumber	UInt32	-	true
ReportEvents	Bool	-	true
SecondaryType ³	String	-	false
StartDate	DateTime	1/1/2012	true
SynchronousApplicationCycleTime	Single	-	true
TimeMode	OBTimeMode	System	true
TimeOfDay	DateTime	12:00 AM	true
TransformationDBNumber	UInt16	0xffff	true
³ When exporting an OB, the "SecondaryType" is additionally set based on the OB number. The assignment is checked during import. If the assignment is incorrect, an exception of type "Recoverable" is thrown.			

Attributes applicable for FB, FC and OB blocks

The following attributes are exported for FB, FC and OB block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
-----------	------	---------------	----------

HandleErrorsWithinBlock	Bool	false	true
-------------------------	------	-------	------

Attributes applicable for FB, FC and UDT blocks

The following attributes are exported for FB, FC and UDT block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
LibraryConformanceStatus	String	-	false

Attributes applicable for GRAPH block

The following attributes are exported for GRAPH block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
AcknowledgeErrorsRequired	Bool	true	false
CreateMinimizedDB	Bool	false	false
ExtensionBlockName	String	-	-
GraphVersion	String	-	false
InitialValuesAcquisition	String	-	-
LanguageInNetworks	String	-	false
LockOperatingMode	Bool	false	false
PermanentILProcessingInMANMode	Bool	false	false
SkipSteps	Bool	false	false

Attributes applicable for GRAPH FB block

The following attributes are exported for GRAPH FB block with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
-----------	------	---------------	----------

WithAlarmHandling	Bool	true	false
-------------------	------	------	-------

Attributes applicable for SCL block

The following attributes are exported for SCL blocks with the selected `ExportOptions`. These attributes are exported based on the type of PLCs.

Attribute	Type	Default value	ReadOnly
CheckArrayLimits	Bool	false	false
ExtendedStatus	Bool	false	false
DBAccessibleFromOPCUA	Bool	true	false

Attributes applicable for GRAPH, SCL, and LAD/FBD blocks

The following attributes are exported for GRAPH, SCL, and LAD/FBD blocks with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
SetENOAutomatically	Bool	-	false

Attributes applicable for program blocks (except OB), DB, and UDT blocks

The following attributes are exported for program blocks (except OB), DB, and UDT blocks under a unit with the selected `ExportOptions`.

Attribute	Type	Default value	ReadOnly
Access	UnitAccessType	Unpublished	false

For Import behavior for the attribute 'Access'

	Imported under unit	Imported not under unit (without SWImportOptions.IgnoreUnitAttributes)	Imported not under unit (with SWImportOptions.IgnoreUnitAttributes)
XML exported			

from under unit	'Access' used and set	RecoverableException is thrown	'Access' ignored
XML exported from not under unit	'Access' gets its default value	'Access' doesn't exist	'Access' doesn't exist

Program code

Modify the following program code to export a block without know-how protection to an XML file:

 Copies the following program code to clipboard.

```
//Exports a regular block

private static void ExportRegularBlock(PlcSoftware
plcSoftware)
{
    PlcBlock plcBlock =
    plcSoftware.BlockGroup.Blocks.Find("MyBlock");

    plcBlock.Export(new FileInfo(string.Format(@"D:\Samples\{ 0
}.xml", plcBlock.Name)), ExportOptions.WithDefaults);
}
```

2.1.14 Importing blocks/UDT with open reference

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open
See Opening a project
- PLC is not online

Introduction

You can use the TIA Portal Openness to import blocks and UDTs even if a related object is missing. You can use the new mode by a new overload of the respective Import method. The new overload has an additional parameter which accepts a value of the new flagged enum SWImportOption

The Openness interface supports the new import mode for the following conditions:

Import of	Object reference
UDT	UDT
DB (global)	UDT
IDBofUDT	UDT
IDBofFB	FB
ArrayDB	Array of UDT
FB	UDT (interface), Multi-instance
FC	UDT (Interface)

Program code

To allow the import, you can use `SWImportOptions.IgnoreMissingReferencedObject`, even if the referenced object is missing.

 Copies the following program code to clipboard.

```
[Flagged] Enum SWImportOptions
{
    None = 0,
    IgnoreStructuralChanges = 1,
    IgnoreMissingReferencedObjects = 2
}

private static void Main(string[] args)
{
    //... // All kinds of blocks
    FileInfo file = ...;
    PlcBlockComposition.Import(file, ImportOptions.None,
    SWImportOptions.IgnoreMissingReferencedObjects);
    //...
    //... // UDTs
```

```
PlcTypeComposition.Import(file, ImportOptions.None,
SWImportOptions.IgnoreMissingReferencedObjects);

//...

}
```

See also

Opening a project

2.1.15 Importing blocks/UDT for structural change object

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open
See Opening a project
- PLC is not online

Application

Using Openness API, you can import blocks and UDTs even if instance data is lost because of a structural change of related objects.

The Openness interface supports the new import mode for the following conditions:

Import of	Object references
Tag	UDT
UDT	UDT
DB (global)	UDT
IDBofUDT	UDT
IDBofFB	FB
ArrayDB	Array of UDT
FB	UDT (interface), Multi-instance
FC	UDT (Interface)

Program code

You can use new mode by a new overload of the respective Import method. The new overload has an additional parameter which accepts a value of the new flagged enum SWImportOptions. To allow the import although there are structural change and maybe a data loss using "SWImportOptions.IgnoreStructuralChanges".

 Copies the following program code to clipboard.

```
Flagged Enum SWImportOptions
{
  None = 0,
  IgnoreStructuralChanges = 1,
  IgnoreMissingReferencedObjects = 2
}
...
// All kinds of blocks
PlcBlockComposition.Import(file, ImportOptions.None,
SWImportOptions.IgnoreStructuralChanges);
...
...
// UDTs
PlcTypeComposition.Import(file, ImportOptions.None,
SWImportOptions.IgnoreStructuralChanges);
...
```

See also

Connecting to the TIA Portal

Opening a project

2.1.16 Export/Import of unit specific publishing attribute of blocks and types

Requirement

- The TIA Openness application is connected to the TIA Portal
See Connecting to the TIA Portal
- A project is open
See Opening a project

Introduction

The Access attribute exists only for blocks, plc types, and tag tables located under a unit. The XML file of block, type, and tag table contains the Access attribute when exporting them into unit

environment and get its Unpublished default value while importing . The exported XML of the same block, type, and tag table does not contain the Access attribute in non-unit environment and gets nothing while importing.

The general Openness XML import rules does not allow to import XMLs containing undefined attributes and undefined values. The rules restrict the export and import of XMLs originated from

object in a unit environment to a non-unit environment.

The rules add limitation in the usage of export and import of XML, therefore the import overloading method is defined which accepts the three parameters

Parameter	Return type	Description
path	String	Specifies the path to the Simatic ML file to be imported
importOptions	enum: Siemens.Engineering.ImportOptions	Specifies the general import option(s) to be used during import.
swImportOptions	enum: Siemens.Engineering.SW.SWImportOptions	Specifies the import option(s) specific to Step7 to be used during import.

The Enum type Siemens.Engineering.SW.SWImportOptions will be extended with the following new import option:

- IgnoreUnitAttributes: Specifies that the import process shouldn't be aborted in case of unit related attributes are present in the XML and the import is performed in a non-unit environment.

The 'IgnoreUnitAttributes' is considered when the XML

- Is exported from a unit
- Contains the 'Access' attribute
- Is imported into non-unit environment

If the exported XML does not contain the Access attribute and it is imported into a unit the new import option will not be considered by the import logic at all.

Program code

 Copies the following program code to clipboard.

```
PlcSoftware plcTarget =  
GetControllerTargetByPLCName(Session.OpnsProject.Devices,  
PLCName);  
  
PlcUnitProvider plcUnitProvider =  
plcTarget.GetService<PlcUnitProvider>();  
  
PlcSoftware plcSoftware =  
plcTarget.GetService<SoftwareContainer>() as PlcSoftware;  
  
PlcUnit plcUnit1 = plcUnitProvider.UnitGroup.Units[0];  
    //assuming Unit_1 is already existing  
PlcUnit plcUnit2 = plcUnitProvider.UnitGroup.Units[1];  
    //assuming Unit_2 is already existing  
  
PlcBlock block1 = plcUnit1.BlockGroup.Blocks.Find("Block_1");  
    //assuming Block_1 is already existing under Unit_1  
PlcBlock block2 = plcUnit2.BlockGroup.Blocks.Find("Block_2");  
    //assuming Block_2 is already existing under Unit_2  
  
PlcBlock block3 =  
plcSoftware.BlockGroup.Blocks.Find("Block_3");  
    //assuming Block_3 is already existing under the PLC
```

Program code: Object exported from a unit and imported into a unit

The examples below are demonstrating the behavior of the new import option by using blocks, the same applies for plc types and tag tables as well.

Modify the following program code to export a block with 'Access' attribute set to value 'Unpublished' (default value) with ExportOptions.WithDefaults and import with SWImportOptions.None:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Unpublished);  
  
block1.Export(new FileInfo("somepath"),  
ExportOptions.WithDefaults);
```

```
block1.Delete();

plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),
ImportOptions.None, SWImportOptions.None);
```

Modify the following program code to export a block with 'Access' attribute set to value 'Unpublished' (default value) with ExportOptions.WithDefaults and import with SWImportOptions.IgnoreUnitAttributes:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Unpublished);
block1.Export(new FileInfo("somepath"),
ExportOptions.WithDefaults);
block1.Delete();

plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),
ImportOptions.None, SWImportOptions.IgnoreUnitAttributes);
```

Modify the following program code to export a block with 'Access' attribute set to value 'Published' with ExportOptions.None and import with SWImportOptions.None:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Published);
block1.Export(new FileInfo("somepath"), ExportOptions.None);
block1.Delete();

plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),
ImportOptions.None, SWImportOptions.None);
```

Modify the following program code to export a block with 'Access' attribute set to value 'Published' with ExportOptions.None and import with SWImportOptions.IgnoreUnitAttributes:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Published);
block1.Export(new FileInfo("somepath"), ExportOptions.None);
block1.Delete();

plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),
ImportOptions.None, SWImportOptions.IgnoreUnitAttributes);
```

Note

In all of the above program codes, No exception is thrown, and import succeeds.

Program code: Object exported from a unit and imported into non-unit environment

Modify the following program code to export a block with 'Access' attribute set to value 'Unpublished' (default value) with ExportOptions.WithDefaults and import with SWImportOptions.None:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Unpublished);  
block1.Export(new FileInfo("somepath"),  
ExportOptions.WithDefaults);  
block1.Delete();  
  
plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),  
ImportOptions.None, SWImportOptions.None);
```

Note

In the above program code, Import does not succeed and a Recoverable Exception is thrown,

Modify the following program code to export a block with 'Access' attribute set to value 'Unpublished' (default value) with ExportOptions.WithDefaults and import with SWImportOptions.IgnoreUnitAttributes:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Published);  
block1.Export(new FileInfo("somepath"), ExportOptions.None);  
block1.Delete();  
  
plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),  
ImportOptions.None, SWImportOptions.None);
```

Note

In the above program code, import succeed and No exception is thrown.

Modify the following program code to export a block with 'Access' attribute set to value 'Published' with ExportOptions.None and import with SWImportOptions.None:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Published);
```

```
block1.Export(new FileInfo(„somepath”), ExportOptions.None);  
block1.Delete();  
plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),  
ImportOptions.None, SWImportOptions.None);
```

Note

In the above program code, Import succeed and a Recoverable exception is thrown.

Modify the following program code to export a block with 'Access' attribute set to value 'Published' with ExportOptions.None and import with SWImportOptions.IgnoreUnitAttributes:

 Copies the following program code to clipboard.

```
block1.SetAttribute("Access", UnitAccessType.Published);  
block1.Export(new FileInfo(„somepath”), ExportOptions.None);  
block1.Delete();  
plcUnit2.BlockGroup.Blocks.Import(new FileInfo("somepath"),  
ImportOptions.None, SWImportOptions.IgnoreUnitAttributes);
```

Note

In the above program code, No exception is thrown and import succeed.

Object exported from non-unit environment into a unit

The exported XML doesn't contain the 'Access' Openness attribute and when importing it gets the default value 'Unpublished'.

Object exported from non-unit environment into non-unit environment

The exported XML doesn't contain the 'Access' Openness attribute and when importing it nothing happens.

2.1.17 Creating Instance DB

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See Connecting to the TIA Portal
- A project is opening
See Opening a project

Introduction

You can use the TIA Portal Openness to create instance DB for programming languages such as SCL, LAD, FBD, STL, Graph, and CEM.

Program code: Creating Instance DB for SCL, LAD, FBD, STL, Graph, & CEM block

 Copies the following program code to clipboard.

```
PlcSoftware plc = ...;

// To create instance DB of SCL block
plc.BlockGroup.Blocks.CreateInstanceDB("ConveyerDB", true, 5,
"Conveyer_SCL_Block");

// To create instance DB of LAD block
plc.BlockGroup.Blocks.CreateInstanceDB("RelayDB", true, 6,
"Relay_LAD_Block");

// To create instance DB of FBD block
plc.BlockGroup.Blocks.CreateInstanceDB("SensorDB", true, 6,
"Sensor_FBD_Block");

// To create instance DB of STL block
plc.BlockGroup.Blocks.CreateInstanceDB("ReadIODB", true, 6,
"ReadIO_STL_Block");

// To create instance DB of Graph block
plc.BlockGroup.Blocks.CreateInstanceDB("ConveyerDB", true, 6,
"Conveyer_Graph_Block");

// To create instance DB of CEM block
plc.BlockGroup.Blocks.CreateInstanceDB("RelayDB", true, 6,
"Relay_CEM_Block");
```

2.1.18 Accessing DB value parameter without export

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See Connecting to the TIA Portal
- You have opened a project via a TIA Portal Openness application
See Opening a project

Introduction

You can use the TIA Portal Openness to read and write the following values of a member attribute in any DB.

Property name	Data type	Modelled/Dynamic	Access
Name	String	Modelled	Read
StartValue	String	Dynamic	Read/Write
AssignedProDiagFB	String	Dynamic	Read
ExternalAccessible	bool	Dynamic	Read/Write
ExternalVisible	bool	Dynamic	Read/Write
ExternalWritable	bool	Dynamic	Read/Write
Retain	bool	Dynamic	Read/Write
SetPoint	bool	Dynamic	Read/Write
DataTypePoint	bool	Dynamic	Read/Write
Snapshot	bool	Dynamic	Read
DefaultValue	bool	Dynamic	Read

Program code

Modify the following program code to access the data block interface member details:

 Copies the following program code to clipboard.

```
public static void PrintMemberAttributes( plcBlockInterface)
{
    foreach(Member member in plcBlockInterface.Members)
    {
        Console.WriteLine($"Name: {member.Name}");
    }
}
```

```

Console.WriteLine($"StartValue:
{member.GetAttribute("StartValue")}");

Console.WriteLine($"ExternalAccessible:
{member.GetAttribute("ExternalAccessible")}");

Console.WriteLine($"ExternalVisible:
{member.GetAttribute("ExternalVisible")}");

Console.WriteLine($"ExternalWritable:
{member.GetAttribute("ExternalWritable")}");

Console.WriteLine($"Retain:
{member.GetAttribute("Retain")}");

Console.WriteLine($"Setpoint:
{member.GetAttribute("Setpoint")}");

Console.WriteLine($"DataTypeName:
{member.GetAttribute("DataTypeName")}");

Console.WriteLine($"Snapshot:
{member.GetAttribute("Snapshot")}");

Console.WriteLine($"DefaultValue:
{member.GetAttribute("DefaultValue")}");
}
}

```

Modify the following program code to read start value from DB:

 Copies the following program code to clipboard.

```

PlcBlockInterface bi = dbbblock.Interface;
MemberComposition members = bi.Members;
Member member = members.Find("Room_Temperature");
string startValue = member.StartValue;

//Normal get attribute should be possible as usual
startValue = member.GetAttribute("StartValue");

//Array
initialSpeedvar = members.Find("Motor.InitialSpeed[0]");
object motorInitialSpeed = initialSpeedvar.StartValue;

//UDT-struct

```

```

axisSpeedvar =
members.Find("FillingStation.Convayer.AxisSpeed");

object axisSpeed = axisSpeedvar.StartValue;
axisSpeed = axisSpeedvar.GetAttribute("StartValue");

//Struct

initialSpeedvar =
paramF.Find("DischargeValve.FlowMeter.InitialSpeed[0]");
object initialSpeed = initialSpeedvar.StartValue;
initialSpeed = initialSpeedvar.GetAttribute("StartValue");

```

Modify the program code to write start value to DB:

 Copies the following program code to clipboard.

```

PlcBlockInterface bi = dbbblock.Interface;
MemberComposition members = bi.Members
Member member = members.Find("Room_Temperature");
member.StartValue = 10.2;

//Normal set attribute should be possible as usual
member.SetAttribute("StartValue", 20.3);

//Array

initialSpeedvar = members.Find("Motor.InitialSpeed[0]");
initialSpeedvar.StartValue = 36;
initialSpeedvar.SetAttribute("StartValue", 56);

//UDT-struct

axisSpeedvar =
members.Find("FillingStation.Convayer.AxisSpeed");
object axisSpeed = axisSpeedvar.StartValue;
axisSpeed.SetAttribute("StartValue", 40.5);

initialSpeedvar =
paramF.Find("DischargeValve.FlowMeter.InitialSpeed[0]");
object initialSpeed = initialSpeedvar.StartValue;
initialSpeed.SetAttribute("StartValue", 12);

```

Modify the following program code to set the start value of a tag to 'myTest':

 Copies the following program code to clipboard.

```
PlcSoftware myPlcSoftware =  
tiaProject.Devices[0].DeviceItems[1].GetService<SoftwareContainer>  
(()).Software as PlcSoftware;  
  
DataBlock myDatablock = myPlcSoftware.BlockGroup.Blocks[1] as  
DataBlock;  
  
myDatablock.Interface.Members[0].SetAttribute("StartValue",  
" 'myTest' ");
```

See also

Opening a project

2.1.19 Export/Import of Plc Alarm TextLists

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See Connecting to the TIA Portal
- A project is open
See Opening a project

Introduction

You can use the TIA Portal Openness to export and import Plc alarm text lists. The format of the Export/Import is XLSX.

Excel file structure

The generated XLSX file consists of two sheets, one for the text lists and other for the entries. The created XLSX file will receive a custom property named FileContent with value "Alarm text lists". If this property is missing or invalid (contains no or different value), the import will be denied.

The data is linked together between the two sheets. The TextListEntry.Parent refers to the TextList.Name

	A	B	C	D	E
1	Name	ListRange	Comment [en-US]	Comment [de-DE]	Comment [fr-FR]
2	NextTextList	Decimal	English	Deutsch	French

A	B	C	D	E	F
Parent	From	To	Text [de-DE]	Text [en-US]	Text [fr-FR]
NextTextList	0	1	xyzxyzxyz	rstrstrst	uvvwuvwuvw

Engineering object model

The Engineering Object Model definition of the newly introduced `PlcAlarmTextListProvider` type. It is accessible both under a `PlcUnit` and `PlcSoftware Engineering` object.

Export

The Two API methods are provided for the export. If all text lists of a PLC/SW Unit in all activated project languages is to be exported, the only required parameter is the 'path' that defines the location where the result XLSX file has to be located after the successful export.

 Copies the following program code to clipboard.

```
FileInfo fileInfo = new
FileInfo(Path.Combine(Environment.CurrentDirectory, $"
{xlsxName}.xlsx"));

PlcAlarmTextListProvider textListProvider =
GetTextListProvider(unitName);

textListProvider.ExportToXlsx(fileInfo);
```

If not all text lists of a PLC/SW Unit has to be exported or the languages - in which the text lists have to be exported - should be filtered, another method is also available in the API, where the list

of text list names identify the text lists to be exported, and the list of Languages identify the languages in which the text lists have to be exported.

Error Handling:

- If the given language is not activated as a project language then the `UserException` will be thrown. For example, "Required language 'Konkani (India)' cannot be exported, because it is not used culture in the project. The valid languages are 'German (Germany)', 'English (United States)'"
- If there is no user text list exist then the `UserException` will be thrown (`TextListNotFoundException`) with the following message: "There is no text list on the following item: <PLCName>."
- If a given text list name does not exist, or exists but not a user text list, a `UserException` will be thrown. Such as "Text list User_1 is not found at PLC_1."
"Text list SYSTEM_SDdiag_CmpCpuName cannot be exported, because it is not user text list."

A sample usage of the `ExportToXlsx` action:

 Copies the following program code to clipboard.

```
private void ExportPlcOrUnit(
string stationName,
```

```

string plcName,
string xlsxName,
string[] textListNames,
string[] cultureCodes,
string unitName = null)
{
    LanguageSettings languageSettings = Project.LanguageSettings;
    LanguageComposition supportedLanguages =
    languageSettings.Languages;
    IEnumerable<Language> cultureInfos = null;
    if (cultureCodes != null)
    {
        cultureInfos = cultureCodes.Select(i =>
        supportedLanguages.Find(CultureInfo.GetCultureInfo(i)));
    }

    FileInfo fileInfo = new
    FileInfo(Path.Combine(Environment.CurrentDirectory, $"
    {xlsxName}.xlsx"));

    PlcAlarmTextListProvider textListProvider =
    GetTextListProvider(stationName, plcName, unitName);
    TextListXlsxResult result = null;
    string exceptionMessage = string.Empty;
    ExceptionMessageData? userExceptionMessageData = null;
    try
    {
        result = textListProvider.ExportToXlsx(fileInfo,
        textListNames, cultureInfos);
    }
    catch (EngineeringTargetInvocationException e)
    {
        exceptionMessage = e.ToString();
        userExceptionMessageData =
        e.DetailMessageData.FirstOrDefault();
    }
}

```

```

}

}

private PlcAlarmTextListProvider GetTextListProvider(string
stationName, string plcName, string unitName = null)
{
    DeviceComposition devices = Project.Devices;

    Device device = devices.Where(station => station.Name ==
stationName).FirstOrDefault();

    if (device == null)
    {
        throw new InvalidOperationException($"The requested
'{stationName}' station is not found");
    }

    PlcSoftware plcTarget =
    (PlcSoftware)FindSoftwareTarget(device, plcName);

    if (plcTarget == null)
    {
        throw new InvalidOperationException($"The requested
'{plcName}' PLC SW is not found")
    }

    if (unitName != null)
    {
        PlcUnitProvider unitProvider =
        plcTarget.GetService<PlcUnitProvider>();

        PlcUnit unit = unitProvider.UnitGroup.Units.Where(unit =>
unit.Name == unitName).FirstOrDefault();

        if (unit == null)
        {
            throw new InvalidOperationException($"The requested
'{unitName}' unit is not found");
        }

        return unit.GetService<PlcAlarmTextListProvider>();
    }
}

```



```
return plcTarget.GetService<PlcAlarmTextListProvider>();
}
```

Import

The following Import action is present at the `PLCAlarmTextListProvider` class. The `ImportFromXlsx` accepts the `fileInfo` path as required parameter that defines the location of imported Xlsx file.

With the `importOptions` parameter as second parameter in the `ImportFromXlsx` that defines whether the import should override existing text lists or not.

If the `importOptions` is set to `ImportOptions.None` and during the import an already existing text list should be updated, a `UserException` will be thrown.

If the `importOptions` parameter is set to `ImportOptions.Override`, then the already existing text lists will be updated during the import.

Existing text lists are identified by their name. New data (text list, entry) will be appended.

Existing text list will not be deleted if there is not any corresponding data in the import file. if

the Excel file contains text list entries for an already existing text list, the already existing entries will be removed, and the entries given in the Excel file will be imported to the TIA project for that specific text list (No merge for text list entries).

A sample usage of the `ImportFromXlsx` action:

 Copies the following program code to clipboard.

```
private TextListXlsxResultState ImportPlcOrUnit( string
xlsxName, string unitName = null)
{
    FileInfo fileInfo = new
    FileInfo(Path.Combine(Environment.CurrentDirectory, $"
{xlsxName}.xlsx"));

    PlcAlarmTextListProvider textListProvider =
    GetTextListProvider(unitName);

    // Definition of GetTextListProvider is at previous
    Export example.

    TextListXlsxResult result = null;

    try
    {
        //Import
```

```

        catch (EngineeringTargetInvocationException e)
        {
            //Critical error case: No data imported. E.g. Missing or invalid x
file.

            System.Diagnostics.Debug.WriteLine(e.Message);

            // Write the place and the reason of the error.

            System.Diagnostics.Debug.WriteLine(e.DetailMessageData.First().Text);

            // Write the the reason of the error only.

            return TextListXlsxResultState.Error;
        }

        if (result.State == TextListXlsxResultState.OK)
        {
            System.Diagnostics.Debug.WriteLine("All Xlsx imported successfully.");
        }

        else if (result.State == TextListXlsxResultState.Warning)
        {
            //Some text list has not imported. E.g. Duplicated row, invalid
range, invalid project language.

            System.Diagnostics.Debug.WriteLine($"Xlsx imported partially. More
details at {result.LogFilePath} file.");
        }

        return result.State;
    }
}

```

If invalid entries exist in the file, the text list containing the invalid entry will not be imported. (Examples for invalid entries: overlap, inconsistent with the containing text list data type.).

The Texts will be imported only in languages, which are active in the project. If the file contains more languages than the project, texts in extra languages will not be imported and a warning will

be logged. If the project contains more languages than the file, texts which have no corresponding data in the file will remain untouched (or empty, if import inserts new text lists or ranges).

No import can be done into a read-only project. A UserException is expected in case the user tries to import into a read-only project.

Import is a complex process, so several kinds of errors can occur. If the error is not critical, the import will be finished and the method returns with a `TextListXlsxResult` object that contains the path to the log file (that contains the messages that are generated during the import), and has a State of `TextListXlsxResultState.Warning`.

If the error is critical a `UserException` will be thrown that contains the details about the error. If the error is fatal, a `NonRecoverableException` will be thrown.

 Copies the following program code to clipboard.

```
<?xml version="1.0" standalone="no"?>

<?xml-stylesheet type='text/xsl' href='MassDataHandlerLogFile.xsl'?>

<LogFile titleName="PLCAAlarmTextLists_InvalidLanguage.xlsx__2019.06.07_1"
projectName="D:\TIA\dev\WM5_WinCC_HW_Work\binaries\Debug\x64\Tests\Siemens"
typeText="Type" messageText="Message" timeText="Time">

<LogEntry type="Warning" dateTime="3:46:45 PM">

<Message>The language ID in column 'Comment [abcd-EF]' is missing or is invalid.</Message>

</LogEntry>

<LogEntry type="Warning" dateTime="3:46:45 PM">

<Message>The language ID in column 'Text [abcd-EF]' is missing or is invalid.</Message>

</LogEntry>

<LogEntry type="Information" dateTime="3:46:45 PM">

<Message>Import completed: 2 text lists with 9 entries.</Message>

</LogEntry>

</LogFile>
```

Note

- XLSX files generated during Export by TIA Portal V16 should have `FileVersion` custom property specified with value 1.
- XLSX files generated during Export by TIA Portal V16 should have `FileContent` custom property specified with value Alarm text lists.
- Log file is generated during Import at the TIA project's Logs folder

Results

The `TextListXlsxResult` object contains information about the result of the export or import. It contains a `FileInfo` named `LogFilePath`, which points to the logfile of the finished process. The

State

(which is of type `TextListXlsxResultState`) will show the final state of the process, which can be OK, Warning or Error. If the result is Error, the process failed, if it is Warning, the process partially succeeded.

2.1.20 Export Document information to SimaticML file

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See [Connecting to the TIA Portal](#)
- A project is open
See [Opening a project](#)

Introduction

You can use the TIA Portal to export `PlcTag`, `PlcTagTable`, and `PlcUserConstant` to SimaticML file with user specific document information. The document information exported to the Simatic ML file from the TIA Portal and imported from the SimaticML file to the TIA Portal is not relevant.

The new export allows you to either ignore full document information while exporting the Openness data types such as `PlcTag`, `PlcTagTable` and `PlcUserConstant` or configure what document information must be available in the SimaticML file.

DocumentInfoOptions

You can use the `Siemens.Engineering.DocumentInfoOptions` to specify the document information you need to export in simaticML file. The `DocumentInfoOptions` is a flag enum that allows you to select multiple configurations. Below are the configurations available for Export:.

Enum	Description
None	No document info is exported
ExportSetting	Export data with only ExportSetting document information, for example it includes the details of ExportOption used for Export.
InstalledProducts	Export data with only InstalledProducts document information, for example it includes product names and option package names

CreatedTimeStamp	Export data with only CreatedTimeStamp document information
All	Export data with all document information (CreatedTimeStamp, ExportSetting, InstalledProducts)

Program code: Export with DocumentInfoOptions



Copies the following program code to clipboard.

```
PlcTagTable tagTable =
FindTagTableToBeExported("TestTagTable");

FileInfo testTagTableExportFile = new
FileInfo($@"E:\temp\TestTagTable.xml");

// No document info will be exported since the configuration
is None.

tagTable.Export(testTagTableExportFile,
ExportOptions.WithDefaults, DocumentInfoOptions.None);

FileInfo testTagExportFile = new
FileInfo($@"E:\temp\TestTag.xml");

PlcTag tag = tagTable.Tags.Find("TestTag");

// CreatedTimeStamp and InstalledProduct will be exported as
DocumentInfo.

tag.Export(testTagExportFile, ExportOptions.WithDefaults,
DocumentInfoOptions.CreatedTimeStamp |
DocumentInfoOptions.InstalledProducts);

FileInfo testUserConstantExportFile = new
FileInfo($@"E:\temp\TestUserConstant.xml");

PlcUserConstant plcUserConstant =
tagTable.UserConstants.Find("TestUserConstant");

// Full Document info will be exported

plcUserConstant.Export(testUserConstantExportFile,
ExportOptions.WithDefaults, DocumentInfoOptions.All);
```

2.1.21 Export/Import of Alarm Instance Text

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal

- A project is open.
See Opening a project

Introduction

You can use the TIA Portal Openness to export and import alarm instance texts which is similar to the export and import of type alarm text form PLC supervisions and alarms editor and also similar to the export/import of instance alarm texts from the CFC alarm editor. The export and import of alarm instance text function via TIA Portal Openness is intended to use for translation purposes. The function does not create any type of alarm instance and any alarm class during import. For example, if the alarm class is identified by name of the excel file exists in the target project, it will be set for the instance alarm during import but non-existing class will not be created.

Excel file structure

	A	B	C	D	E	F
1	Location	Alarm name	Alarm class	"Alarm text" - English (United States) / [en-US] / Event text	Fieldinfo / "Alarm text" / Event text	"Info text" - Eng
2	Datenbaustein_1	Static_1	### Inherited from Type ###	alarm text 1		English Info Inst
3						
4						
5						

Inherited from type### is a special content which instructs the system to use the text or the Alarm class of the type. The Alarm class field refers to an existing alarm class name in the target project. Important rule that for a particular alarm text, either all or no language fields should contain ###Inherited from type###. If there are some mixed content fields, no texts for that alarm will be imported at all and a warning will be issued.

Export function takes care about the consistency of the file data.

Freely over-writable fields are the language-dependent ones, i.e. fields which headers contain language identifiers, e.g. "Alarm text" - English (United States) / [en-US] / Event text.

The file also contains some special properties to let the system make some checks prior to import. If these properties are invalid, then import cannot be made.

Export

The following action is provided for export at the PlcAlarmTextProvider.

- PlcAlarmTextProvider

The following parameters are provided for import at the PlcAlarmTextProvider:

- Path
- Language
- Option

The export can be a PLC or a Software Unit. Path parameter is mandatory, Languages can be empty or null. In this case texts in all active project languages will be exported. Option parameter is also mandatory. It takes value(s) of PlcAlarmTextXlsxExportOption enum.

PlcAlarmTextXlsxExportOption has the following values:

- None (Only Alarm texts will be exported)
- IncludeInfoText (Also Info texts will be exported)
- IncludeAdditionalTexts (Also additional text 1 - 9 will contained in the export)
- IncludeAlarmClass (Alarm class setting will be added to the export)
- All

The above three Include options can be combined, for example. if you want Alarm text, Additional texts and Alarm class but not the Info text, then we set IncludeAdditionalTexts and IncludeAlarmClass.

If an error occurs, UserException will be thrown. These cases could be, for example, inactive or non existing languages or file IO errors. Message for invalid language will be in the following format:

"Required language 'Konkani (India)' cannot be exported, because it is not used culture in this project. Valid languages are 'German (Germany)', 'English (United States)'"

Import

The following action is provided for import at the PlcAlarmTextProvider:

- ImportInstanceTextsToXlsx

The following parameters are provided for import at the PlcAlarmTextProvider:

- Path
- Language

The import can be a PLC or a Software Unit. Path parameter is mandatory, Languages can be empty or null. In this case texts in all active project languages will be imported. (If the excel file contains languages which are inactive in the project, those languages will not be activated and no import will be done in those languages.)

 Copies the following program code to clipboard.

```

FileInfo fileInfo = new
FileInfo(Path.Combine(s_XlsxFilesFolderName, $"
{xlsxName}.xlsx"));

List<Language> cultureInfos = new List<Language>();

PlcSoftware plcTarget = ...;

PlcAlarmTextProvider alarmTextsProvider =
plcTarget.GetService<PlcAlarmTextProvider>();

PlcAlarmTextXlsxResult result =
alarmTextsProvider.ExportInstanceTextsToXlsx(fileInfo,
cultureInfos, PlcAlarmTextXlsxExportOption.All);

//Project project = ...;
```

```

LanguageComposition supportedLanguages = languageSettings.Languages;
cultureInfo =
cultureInfos.Add(supportedLanguages.Find(CultureInfo.GetCultureInfo("en-
US")));

PlcAlarmTextProvider alarmTextsProvider =
plcTarget.GetService<PlcAlarmTextProvider>();

result = alarmTextsProvider.ImportInstanceTextsFromXlsx(fileInfo,
cultureInfo);

```

Result

Information about the result of the export or import will be provided in a `PlcAlarmTextXlsxResult` object. It contains a `FileInfo` structure as `LogFilePath`, which points to the logfile of the finished process. The State (which is of type `PlcAlarmTextXlsxResultState`) will show the final state of the process. Values can be OK, Warning or Error. If the result is Error, the process failed, if it is Warning, the process partially succeeded.

Exception

Non recoverable exceptions

These are technical issues, none of them should appear for you.

- "Missing PlcAlarmTextProvider"
- "Plc/SW Unit cannot be retrieved"
- Parameter has not found. Please contact Siemens technical support.
- "ProjectService is not available"
- path Parameter has not found. (Parameters: nnn, mmm) Please contact Siemens technical support.
- Result State is not error, but there are some error messages: xyz.

Recoverable exceptions

These could appear to you due to e.g improper parameter values.

- Required language 'xx-XX' cannot be exported, because it is not set for this PLC. Valid languages are yy-YY, zz-ZZ.

Various system error messages can also appear, e.g. file IO errors, etc.

See also

Opening a project

2.1.22 Export/Import of Alarm classes

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness to export/import Alarm classes. The format of the Export/Import is .DAT.

Engineering Object Model

The Engineering object model definition of the newly introduced `AlarmClassDataProvider` type. It is accessible under the `ProjectBase Engineering` object.

Export

The following action is present at the `AlarmClassDataProvider` class for Export:

The export action has one parameter: `path`.

With the `path` parameter the file path can be given where the result of the export is to be placed. The type of the parameter is `FileInfo`.

The result of the action is an `AlarmClassExportImportResult` object. It contains the count of errors, warnings and the state of the export result (Success, Warning or Error).

It also has a `Messages` composition property, that contains the messages generated during the export together with the state (Success, Error or Warning).

A sample usage of the Export action:

 Copies the following program code to clipboard.

```
FileInfo fileInfo = new FileInfo(@"D:\AlarmClasses.DAT");
AlarmClassDataProvider provider =
Project.GetService<AlarmClassDataProvider>();
if (fileInfo.Exists)
{
fileInfo.Delete();
}
AlarmClassExportImportResult result;
result = provider.Export(fileInfo);
```

```

Assert.AreEqual(AlarmClassExportImportResultState.Success,
result.State);

Assert.AreEqual(0, result.ErrorCount);

Assert.AreEqual(0, result.WarningCount);

AlarmClassExportImportResultMessageComposition messages =
result.Messages;

Assert.AreEqual(1, messages.Count);

Assert.AreEqual($"Export of Alarm class settings file
'{fileInfo.FullName}' was successful.",
messages.FirstOrDefault().Message);

Assert.AreEqual(AlarmClassExportImportResultState.Success,
messages.FirstOrDefault().State);

Assert.IsTrue(File.Exists(fileInfo.FullName), "Exported file
does not exist.");

fileInfo.Delete();

```

Error Handling:

- If the overall state of the export process is Error a `UserException` is raised, and the list of error messages is stored in the exception itself.

From client side an `EngineeringTargetInvocationException` exception can be caught, where the `DetailMessageData` property contains the error messages.

- Possible errors:
 - IO error during the export process

Import

The import action is similar to the export action.

A sample of usage of the Import action:

 Copies the following program code to clipboard.

```

FileInfo fileInfo = new FileInfo(@"D:\AlarmClasses.DAT");
AlarmClassDataProvider provider =
Project.GetService<AlarmClassDataProvider>();

AlarmClassExportImportResult result;

result = provider.Import(fileInfo);

Assert.AreEqual(AlarmClassExportImportResultState.Success,
result.State);

```

```

Assert.AreEqual(0, result.ErrorCount);
Assert.AreEqual(0, result.WarningCount);
AlarmClassExportImportResultMessageComposition messages =
result.Messages;
Assert.AreEqual(1, messages.Count);
Assert.AreEqual($"Import of Alarm class settings file
'{fileInfo.FullName}' was successful.",
messages.FirstOrDefault().Message);
Assert.AreEqual(AlarmClassExportImportResultState.Success,
messages.FirstOrDefault().State);

```

Error handling:

- If the overall state of the export process is Error a `UserException` is raised, and the list of error messages is stored in the exception itself.

From client side an `EngineeringTargetInvocationException` exception can be caught, where the `DetailMessageData` property contains the error messages

- Possible errors
 - File to be imported has invalid file extension
 - File to be imported missing file extension
 - With the import the count of alarm classes would exceed the maximum allowed number of alarm classes in the project
 - The file to be imported uses not the correct schema version (Only version 1.0 and 2.0 is supported)
 - No overlapping between the activated project languages and the languages given in the import file
 - No language is defined
 - IO error during the export process

Note

It shall be supported to import files generated during Export with TIA Portal V16.

Result

The `AlarmClassExportImportResult` object will contain an Information about the result of the export or import. It contains an `AlarmClassExportImportResultMessageComposition` navigator named `Messages`, which points to the messages generated during the export and import process. The `State` (which is of type `AlarmClassExportImportResultState`) will show the final state

of the process, which can be Success, Warning or Error. If the result is Error, the process failed, if it is Warning, the process succeeded.

See also

Opening a project

2.1.23 Export/Import of global supervision for ProDiag-FB

Requirement

- The TIA Portal Openness application is connected to the TIA Portal
See Opening a project
- A project is open
See Opening a project

Introduction

You can use the TIA Portal Openness to support export/import of global supervisions of ProDiag block.. This functionality is supported by service provider mechanism and the same service provider is used by the block under Plc and software unit to perform export and import operations.

On performing export/import you shall be communicated with proper result state or with proper user exceptions.

The following method can be used to export and import global supervisions of ProDiag block:

Method	Description
ExportSupervisionsToXlsx(FileInfo path)	Export the supervisions tags including settings in the TIA portal supported format and generates the results of the export
ImportSupervisionsFromXlsx(FileInfo path, ImportOptions importOptions)	Import supervision tags with supported import options
ImportSupervisionSettingsFromXlsx(FileInfo path, ImportOptions importOptions)	Import supervision settings which are placed in exported file along with supervised tags

The following properties are available under SupervisionXlsxResult:

Property Name	Data Type	Access
---------------	-----------	--------

LogFilePath	System.IO.FileInfo	Read
State	SupervisionXlsxResult	Read

The SupervisionXlsxResultState has the following enum values:

ENUM	Values
SupervisionXlsxResultState	Success
	Failure

Program code

Modify the following program code to export global supervisions of ProDiag block:

 Copies the following program code to clipboard.

```
// File Path for the export
FileInfo fileInfo = new
FileInfo(@"C:\Users\z003jwfc\Desktop\Supervisions_Openness.Xlsx");
//SW is nothing but PlcSoftware / PlcUnit.
var proDiagBlock = (FB) SW.BlockGroup.Blocks.Find("Block1");
SupervisionProvider supervisionProvider =
proDiagBlock.GetService<SupervisionProvider>();
SupervisionXlsxResult result =
supervisionProvider.ExportSupervisionsToXlsx(fileInfo);
```

Modify the following program code to import global supervisions of ProDiag block:

 Copies the following program code to clipboard.

```
// File Path for the import
FileInfo fileInfo = new
FileInfo(@"C:\Users\z003jwfc\Desktop\SupervisionsOpenness.Xlsx");
//SW is nothing but PlcSoftware / PlcUnit.
var proDiagBlock = (FB) SW.BlockGroup.Blocks.Find("Block1");
SupervisionProvider supervisionProvider =
proDiagBlock.GetService<SupervisionProvider>();
//import supervisions
```

```

SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionsFromXlsx(fileInfo,
ImportOptions.None);

SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionsFromXlsx(fileInfo,
ImportOptions.Override);

//import supervision settings
SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionSettingsFromXlsx(fileInfo,
ImportOptions.None);

SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionSettingsFromXlsx(fileInfo,
ImportOptions.Override);

```

2.1.24 Export/import of ProDiag supervisions from Global overview editor

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- Opening a project
See Opening a Project

Introduction

You can use the TIA Portal Openness to support export and import of global supervisions available under Plc or under Unit. The functionality is supported by service provider mechanism and the same service provider is used mainly by both Plc and software unit to perform export and import operations. On performing export/import, you shall be communicated with proper result or with exceptions.

The following method can be used to export and import global supervisions:

Method	Description
ExportSupervisionsToXlsx(FileInfo path)	Export the supervisions tags including settings in the TIA portal supported format and generates the results of the export
ImportSupervisionsFromXlsx(FileInfo path, ImportOptions importOptions)	Import supervision tags with supported import options

ImportSupervisionSettingsFromXlsx(FileInfo path,ImportOptions importOptions)	Import supervision settings which are placed in exported file along with supervised tags
--	--

The following properties are available under SupervisionXlsxResult:

Property name	Data type	Access
LogFilePath	System.IO.FileInfo	Read
State	SupervisionXlsxResult	Read

The SupervisionXlsxResultState has the following ENUM values:

ENUM	Values
SupervisionXlsxResultState	Success
	Failure

Introduction

Modify the following program code to export global supervisions:

 Copies the following program code to clipboard.

```
//File Path for the export
FileInfo fileInfo = new
FileInfo(@"C:\Users\z003jwfc\Desktop\Supervisions_Openness.Xlsx");
//SW is nothing but PlcSoftware / PlcUnit.
SupervisionProvider supervisionProvider =
SW.GetService<SupervisionProvider>();
SupervisionXlsxResult result =
supervisionProvider.ExportSupervisionsToXlsx(fileInfo);
```

Modify the following program code to import global supervisions:

 Copies the following program code to clipboard.

```
//File Path for the import.
FileInfo fileInfo = new
FileInfo(@"C:\Users\z003jwfc\Desktop\Supervisions_Openness.Xlsx");
```

```
//SW is nothing but PlcSoftware / PlcUnit.
Supervision Provider supervisionProvider =
SW.GetService<SupervisionProvider>();

//import supervisions
SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionsFromXlsx(fileInfo,
ImportOptions.None);

SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionsFromXlsx(fileInfo,
ImportOptions.Override);

//import supervision settings
SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionsSettingsFromXlsx(fileInfo,
ImportOptions.None);

SupervisionXlsxResult result =
supervisionProvider.ImportSupervisionsSettingsFromXlsx(fileInfo,
ImportOptions.Override);
```

See also

Opening a project

2.1.25 Export/Import of settings for ProDiag supervisions

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project

Introduction

You can use the TIA Portal Openness to support export/import of supervision settings.

The following properties are available under SupervisionSettingsExportImportResult:

Property Name	Data Type	Access
State	SupervisionSettingExportImportResultState	Read

ErrorCount	Int32	Read
WarningCount	Int32	Read
Message	String	Read

The ENUM SupervisionSettingExportImportResultState has following value:

ENUM	Value
SupervisionSettingExportImportResultState	Success
	ErrorRollback

Program code

Modify the following program code to export supervision settings:

 Copies the following program code to clipboard.

```
// MyProject obtains SupervisionSettingsProvider
SupervisionSettingsProvider settingsProvider =
MyProject.GetService<SupervisionSettingsProvider>();
// Import file path defined
FileInfo exportFile = new
FileInfo(@"D:\Temp\ProDiagSettings.dat");
// Stores result after import
SupervisionSettingsExportImportResult exportResult =
settingsProvider.Export(exportFile);
// Result state after import
SupervisionSettingsExportImportResultState resultState =
exportResult.State;
// Total error count of result messages
int totalErrorCount = exportResult.ErrorCount;
// Composition of result messages
SupervisionSettingsExportImportResultMessageComposition
exportMessageList = exportResult.Messages;
// Count of result messages
```

```
exportMessageList.Count;

// Read out specific message text from each message
exportMessageList [0].Message;

// Output: "Export of file 'D:\temp\SupervisionSettings.dat' with
the ProDiag supervision settings was successful.";
```

Modify the following program code to import supervision settings:

 Copies the following program code to clipboard.

```
// MyProject obtains SupervisionSettingsProvider
SupervisionSettingsProvider settingsProvider =
MyProject.GetService<SupervisionSettingsProvider>();

// Import file path defined
FileInfo importFile = new
FileInfo(@"D:\Temp\ProDiagSettings.dat");

// Stores result after import
SupervisionSettingsExportImportResult importResult =
settingsProvider.Import(importFile);

// Result state after import
SupervisionSettingsExportImportResultState resultState =
importResult.State;

// Total error count of result messages
int totalErrorCount = importResult.ErrorCount;

// Composition of result messages
SupervisionSettingsExportImportResultMessageComposition
importMessageList = importResult.Messages;

// Count of result messages
importMessageList.Count;

// Read out specific message text from each message
importMessageList[0].Message;

// Output: "Import of the file 'D:\temp\SupervisionSettings.dat'
with the ProDiag supervision settings was successful.";
```

See also

Connecting to the TIA Portal

Opening a project

2.1.26 Export/Import Watch & Force Table

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- You have opened a project with your TIA Portal Openness application.
See Opening a project

Introduction

You can use the TIA Portal Openness to export Watch Table and Force Table from TIA Portal to SIMATIC ML and then import Watch Table and Force Table from SIMATIC ML.

The Export option in Watch Table can be set with the following definition (None, WithDefaults, WithReadOnly, WithDefaultsAndReadOnly). A WatchTable has only one published property i.e. the name. The name is published for read.

The imported WatchTable will be added to the list of WatchTables. The Import options in Watch Table should be set to the required one (None, Override). The ForceTables can be imported in a similar way, but it is only allowed to have one ForceTable.

If you use the None option instead of Override and a (non-empty) ForceTable already exists, then the import will fail with the RecoverableException: Only one ForceTable is allowed to be imported.

Program code: Export Watch Table and Force Table

Modify the following program code to export Watch Table:

 Copies the following program code to clipboard.

```
SoftwareContainer softwareContainer =  
( (IEngineeringServiceProvider) item ).GetService<SoftwareContainer>  
( );  
  
PlcSoftware plcSoftware = softwareContainer.Software as  
PlcSoftware;  
  
PlcWatchTableComposition exportWatchTables =  
plcSoftware.PlcWatchAndForceTableGroup.WatchTables;  
  
PlcWatchTable watchTable =  
exportWatchTables.Find(watchTableName);  
  
if (watchTable != null)  
{  
  
    watchTable.Export((FileInfo) fileInfo, ExportOptions.None);  
}
```

```
}
```

Note

Export options in Watch Table should be set (None, WithDefaults, WithReadOnly, WithDefaultsAndReadOnly.). A WatchTable has only one published property i.e. the name. The name is published for read.

Modify the following program code to export Force Table:

 Copies the following program code to clipboard.

```
SoftwareContainer softwareContainer =  
( (IEngineeringServiceProvider) item) .GetService<SoftwareContainer>  
( );  
  
PlcSoftware plcSoftware = softwareContainer.Software as  
PlcSoftware;  
  
PlcForceTableComposition exportForceTables =  
plcSoftware.PlcWatchAndForceTableGroup.ForceTables;  
  
PlcForceTable forceTable = exportForceTables[0];  
forceTable.Export((FileInfo) fileInfo, ExportOptions.None);
```

Note

There is only one ForceTable in every situation, and it has a read-only name.

Program code: Import Watch Table and Force Table

Modify the following program code to import Watch Table:

 Copies the following program code to clipboard.

```
SoftwareContainer softwareContainer =  
( (IEngineeringServiceProvider) item) .GetService<SoftwareContainer>  
( );  
  
PlcSoftware plcSoftware = softwareContainer.Software as  
PlcSoftware;  
  
PlcWatchTableComposition importWatchTables =  
plcSoftware.PlcWatchAndForceTableGroup.WatchTables;  
  
IList<PlcWatchTable> WatchTables =  
importWatchTables.Import((FileInfo) fileInfo, ImportOptions.None);
```

Note

The imported WatchTable will be added to the list of WatchTables. Import options in Watch Table should be set to the required one (None, Override).

ForceTables can be imported in a similar way, but it is only allowed to have one ForceTable. If you use the None option instead of Override and a (non-empty) ForceTable already exists, then the import will fail with the following RecoverableException: Only one ForceTable is allowed to be imported. Please use Override importOption to overwrite the existing one.

2.1.27 Exporting user data type

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project
- PLC is not online

Program code

Modify the following program code to export an user data type to an XML file:

 Copies the following program code to clipboard.

```
//Exports a user defined type
private static void ExportUserDefinedType(PlcSoftware
plcSoftware)
{
    string udtname = "udt name XYZ";
    PlcTypeComposition types = plcSoftware.TypeGroup.Types;
    PlcType udt = types.Find(udtname);
    udt.Export(new
FileInfo(string.Format(@"C:\OpennessSamples\udts\{0}.xml",
udt.Name)), ExportOptions.WithDefaults);
}
```

2.1.28 Importing user data type

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
Connecting to the TIA Portal
- A project is open.
Opening a project
- PLC is not online.

Introduction

The API interface supports the importing of user data types from an XML file.

Import file syntax

The following code example shows an excerpt from an import file of a user-defined data type:

 Copies the following program code to clipboard.

```
<Section Name="Input">
  <Member Name="Input1" Datatype="myudt1">
    <Sections>
      <Section Name="None">
        <Member Name="MyUDT1Member1" Datatype="bool"/>
        <Member Name="MyUDT1Member2" Datatype="myudt1">
          <Sections...>
```

Note

Syntax for user-defined data types of elements

An exception is thrown if the user-defined data type of an element in the import file for user data types has incorrect syntax.

Make sure that user-defined data types are noted with `"`.

Program code

Modify the following program code to import a user data type:

 Copies the following program code to clipboard.

```
//Imports user data type
private static void ImportUserDataTypes(PlcSoftware
plcSoftware)
{
```

```

        FileInfo fullFilePath = new
FileInfo(@"C:\OpennessSamples\Import\ExportedPlcType.xml");

        PlcTypeComposition types = plcSoftware.TypeGroup.Types;

        IList<PlcType> importedTypes = types.Import(fullFilePath,
ImportOptions.Override);
    }

```

See also

Importing configuration data

2.1.29 Exporting program block as document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness application to export and import program block and DBs components as a document. You can access the components of your TIA Portal project using the TIA Portal Openness. By using files in SIMATIC SD format, you can directly read the program code of the controller. For information on import program block from document, refer [Importing program block from document](#)

The `ExportAsDocument()` method is used to export program block and DBs as a document. The method allows you to specify the target directory and the base file path without extension to export as a document. The result of the export operation is available in the `DocumentExportResult` object, which provides details about the exported files, the state of the export operation, and any messages logged during the process.

Parameter

The `ExportAsDocument()` method accepts the following parameter while using the TIA Portal Openness application:

Parameter name	Data type	Description
directoryInfo	System.IO.DirectoryInfo	Specifies the directory where the documents will be saved. The directory

		must exist before calling this method.
fileNameWithoutExtension	String	Specifies the base file name for the exported documents.

Return type

The ExportAsDocument() method returns the DocumentExportResult, which is used to store the result of exported document. The return type DocumentExportResult includes the following components:

Name	Data type	Description
ImportedPlcBlocks	Siemens.Engineering.SW.PlcBlockAssociation	List of program blocks which are exported.
Messages	DocumentResultMessageComposition	Returns the messages which are logged during export operation
Parent	IEngineeringObject	EOM parent of this object
State	DocumentResultState	Status of document import operation

Program code

To export a PLC block as documents modify the following program code:

 Copies the following program code to clipboard.

```
private void ExportPLCBlockAsDocument(PlcSoftware
plcSoftware)
{
    DocumentExportResult documentExportResult =
plcSoftware.BlockGroup.Blocks[0].
```



```

        ExportAsDocuments(new DirectoryInfo(@"D:\Project\Exported
Files"), "LAD_Block");

        DocumentResultState documentResultState =
documentExportResult.State;

        IEnumerable<FileInfo> exportedDocuments =
documentExportResult.ExportedDocuments;

        DocumentResultMessageComposition documentMessages=
documentExportResult.Messages;

        // DocumentResultMessageComposition contains list of
DocumentResultMessage.

        DocumentResultMessage documentResultMessage =
documentExportResult.Messages[0].Message;
    }

```

2.1.30 Importing program block from document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project

Introduction

You can use the TIA Portal Openness application to import program block and DBs components from document files within specified directory . The ImportFromDocuments() method is used to import PLC blocks from document format directly into the TIA Portal project.

Parameter

The ImportFromDocument() method accepts the following parameter while using the TIA Portal Openness application:

Parameter name	Data type	Description
directoryInfo	System.IO.DirectoryInfo	The directory containing the document files for import.
fileNameWithoutExtension	String	The name of the file to be imported, without its

		extension
ImportDocumentOptions	Siemens.Engineering.SW.ImportDocumentOptions	Options to customize the import process, such as handling conflicts or overrides using the following enum values: • None • Override • Skipinactiveculture • ActiveInactiveculture

Return Type

The ImportFromDocument() method returns the DocumentImportResultForBlocks, which is used to store the result of imported document. The return type DocumentImportResultForBlock includes the following components:

Name	Data type	Description
ImportedPlcBlocks	Siemens.Engineering.SW.PlcBlockAssociation	Collection of imported Plc Blocks.
Messages	Messages	Returns the messages which are logged during import operation.
Parent	IEngineeringObject	The parent engineering object (IEngineeringObject) associated with this operation.IEngineeringObject.
State	DocumentResultState	The status of the document import operation.

Program code

To import program block from document modify the following program code:

 Copies the following program code to clipboard.

```

private void ImportProgramBlockFromDocument(PlcSoftware
plcSoftware)
{
    DocumentImportResult documentImportResultForBlocks =
plcSoftware.BlockGroup.Blocks.

    ImportFromDocuments(new
DirectoryInfo(@"D:\Project\Exported Files"),
    " LAD_Block", ImportDocumentOptions.Override);

    DocumentResultState documentResultState =
documentImportResultForBlocks.State;

PlcBlockAssociation importedPlcBlocks =
documentImportResultForBlocks.ImportedPlcBlocks;

    DocumentResultMessageComposition documentMessages =
documentImportResultForBlocks.Messages;

    // DocumentResultMessageComposition contains list of
DocumentResultMessage.

    DocumentResultMessage documentResultMessage =
documentImportResultForBlocks.Messages[0].Message;
}

```

2.1.31 Exporting UDT as document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See [Connecting to the TIA Portal](#)
- A project is open.
See [Opening a project](#)

Introduction

You can use the TIA Portal Openness application to export and import UDT as a document. The `ExportAsDocument()` method is used to export UDT as a document. The method allows you to specify the target directory and the base file path without extension to export UDT as a document. The result of the export operation is available in the `DocumentExportResult` object, which provides details about the exported files, the state of the export operation, and any messages logged during the process. For information on importing UDT as document, refer [Importing UDT from document](#)

Parameter

The ExportAsDocument() method accepts the following parameter while using the TIA Portal Openness application:

Parameter name	Data type	Description
directoryInfo	System.IO.DirectoryInfo	Specifies the directory where the documents will be saved. The directory must exist before calling this method.
fileNameWithoutExtension	String	Specifies the base file name for the exported documents.

Return type

The ExportAsDocument() method returns the DocumentExportResult, which is used to store the result of exported document. The return type DocumentExportResult includes the following components:

Name	Data type	Description
ExportedDocuments	Siemens.Engineering.SW.PlcBlockAssociation	List of documents which are exported
Messages	DocumentResultMessageComposition	Returns the messages which are logged during export operation.
Parent	IEngineeringObject	EOM parent of this object.
State	DocumentResultState	Status of document

		export operation.
--	--	-------------------

Program code

To export UDT as a document modify the following program code:

 Copies the following program code to clipboard.

```
private void ExportUDTAsDocument(PlcSoftware plcSoftware)
{
    DocumentExportResult documentExportResult =
    plcSoftware.TypeGroup.Types[0].
        ExportAsDocuments(new
    DirectoryInfo(@"D:\Project\Exported Files"),
    "fileNameWithoutExtension");

    DocumentResultState documentResultState =
    documentExportResult.State;

    IEnumerable<FileInfo> exportedDocuments =
    documentExportResult.ExportedDocuments;

    DocumentResultMessageComposition documentMessages- =
    documentExportResult.Messages;

    // DocumentResultMessageComposition contains list of
    DocumentResultMessage.

    DocumentResultMessage documentResultMessage =
    documentExportResult.Messages[0].Message;
}
```

2.1.32 Export of data in OPC UA XML format

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project
- PLC is not online

Application

You can use the TIA Portal Openness to export PLC data as OPC UA XML file. As input parameter for the action you need an absolute directory path, where the xml file will be saved.

Program code

Modify the following program code to export PLC data as OPC UA XML file:

 Copies the following program code to clipboard.

```
//Export PLC data as OPC UA XML file

private static void OpcUaExport(Project project, DeviceItem
plc)
{
    OpcUaExportProvider opcUaExportProvider =
    project.HwUtilities.Find("OPCUAExportProvider") as
    OpcUaExportProvider;

    if (opcUaExportProvider == null) return;

    opcUaExportProvider.Export(plc, new
    FileInfo(string.Format(@"D:\OPC UA export files\{0}.xml",
    plc.Name)));
}
```

2.1.33 Importing UDT from document

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
See Connecting to the TIA Portal
- A project is open.
See Opening a project

Introduction

You can use the TIA Portal Openness to import UDT as a document. The Import FromDocument() method is used to export UDT as a document. The method allows you to specify the target directory and the base file path without extension to export UDT as a document. The result of the export operation is available in the DocumentExportResult object, which provides details about the exported files, the state of the export operation, and any messages logged during the process.

Parameter

The ImportFromDocument() method accepts the following parameter while using the TIA Portal Openness application:

Parameter name	Data type	Description
directoryInfo	System.IO.DirectoryInfo	The directory containing the document files for import.
fileNameWithoutExtension	String	The name of the file to be imported, without its extension
ImportDocumentOptions	Siemens.Engineering.SW.ImportDocumentOptions	Options to customize the import process, such as handling conflicts or overrides using the following enum values: • None • Override • Skipinactiveculture • ActiveInactiveculture

Return type

The ImportFromDocument() method returns the DocumentImportResultForBlocks, which is used to store the result of imported document. The return type DocumentImportResultForBlock includes the following components:

Name	Data type	Description
ImportedPlcBlocks	Siemens.Engineering.SW.PlcBlockAssociation	Collection of imported UDT.
Messages	Messages	Returns the messages which are logged during import operation.

Parent	IEngineeringObject	The parent engineering object (IEngineeringObject) associated with this operation.IEngineeringObject.
State	DocumentResultState	The status of the document import operation.

Program code

To import UDT from document modify the following program code:

 Copies the following program code to clipboard.